

운영체제 3

남궁명수

[1-1] 가상화: 페이징_개요	3
[1] 간단한 예제 및 개요	3
[2] 페이지 테이블은 어디에 저장되는가	8
[3] 페이지 테이블에는 실제 무엇이 있는가	9
[4] 페이징: 너무 느림	12
[5] 메모리 트레이스	15
[1-2] 가상화: 더 빠른 변환(TLB)	18
[1] TLB의 기본 알고리즘	19
[2] 예제: TLB와 배열 접근 예제	20
[3] TLB 미스처리: 하드웨어 vs 소프트웨어	23
[4] 하드웨어 TLB 구성과 연관 방식	25
[5] TLB와 문맥 교환 문제	26
[6] 이슈: 교체 정책	28
[7] 실제 TLB	29
[1-3] 가상화: 페이징_더 작은 테이블	31
[1] 간단한 해법: 더 큰 페이지	31
[2] 하이브리드 접근 방법: 페이징과 세그먼트	36
[3] 멀티 레벨 페이지 테이블	40
[4] 역 페이지 테이블	50
[5] 페이지 테이블을 디스크로 스와핑하기	50
[1-4] 가상화: 물리 메모리 크기의 극복_메커니즘	51
[1] 스왑 공간	52
[2] Present Bit	53
[3] 페이지 폴트	55
[4] 메모리에 빈 공간이 없으면?	56
[5] 페이지 폴트의 처리	57
[6] 교체는 실제 언제 일어나는가	59
[1-5] 가상화: 물리 메모리 크기의 극복_정책	60
[1] 캐시 관리	60
[2] 최적 교체 정책	62
[3] 간단한 정책: FIFO	65
[4] 또 다른 간단한 정책: 무작위 선택	66
[5] 과거 정보의 사용: LRU	67
[6] 워크로드에 따른 성능 비교	69
[7] 과거 이력 기반 알고리즘의 구현	72

[8] LRU 정책 근사하기	73
[9] 갠신된 페이지(Dirty Page)의 고려	75
[10] 다른 VM 정책들	76
[11] 쓰래싱(Thrashing)	77
[1-7] 가상화: VAX/VMS 가상 메모리 시스템	78
[1] 배경: 범용 시스템의 딜레마	78
[2] 메모리 관리 하드웨어	79
[3] 실제 주소 공간 설계의 특징	82
[4] 페이지 교체: 하드웨어 한계 극복(VMS)	86
[5] 성능 최적화: Lazy 전략	89

[1-1] 가상화: 페이징_개요

핵심 질문: 페이지를 사용하여 어떻게 메모리를 가상화할 수 있을까
세그멘테이션의 문제점을 해결하기 위해 페이지를 사용하여 어떻게 메모리를
가상화할 수 있는가?
기본적인 기법은 무엇인가?
공간과 시간 오버헤드를 최소로 하면서 그 기법을 작동작하게 만들기 위한 방법은
무엇인가?

운영체제는 거의 모든 공간 관리 문제를 해결할 때 두 가지 중 하나를 사용한다.
첫 번째 방법은 우리가 가상 메모리의 세그멘테이션에서 보았듯이, 가변 크기의
조각들로 분할하는 것이다.

불행하게도, 이 해결책은 태생적인 문제를 가지고 있다.

공간을 다양한 크기의 청크로 분할할 때 공간 자체가 단편화될 수 있고, 할당은 점점
더 어려워진다.

두 번째 방법은 공간을 동일 크기의 조각으로 분할하는 것을 고려해 볼 필요가 있다.
가상 메모리에서 이를 페이징이라 부르고 이 아이디어의 태생은 초기의 중요한
시스템인 Atlas 까지 거슬러 올라간다.

프로세스의 주소 공간을 몇 개의 가변 크기의 논리 세그먼트(예: 코드, 힙, 스택)로
나누는 것이 아니라 고정 크기의 단위로 나눈다.

이 각각의 고정 크기 단위를 페이지(page)라고 부른다.

상응하여 물리 메모리도 페이지 프레임(page frame)이라고 불리는 고정 크기의
슬롯의 배열이라고 생각한다.

이 프레임 각각은 하나의 가상 메모리 페이지를 저장할 수 있다.

[1] 간단한 예제 및 개요

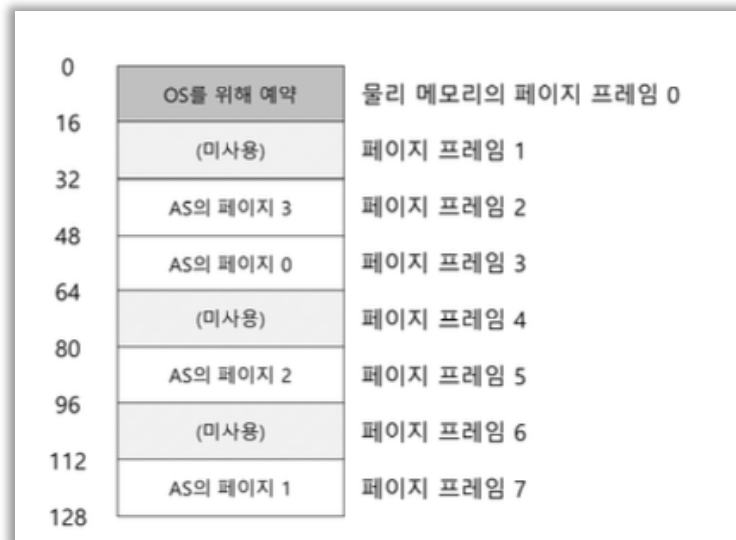
이 방법을 명확히 이해하기 위해 간단한 예를 통해 설명해보자.

총 크기가 64 바이트이면서 4 개의 16 바이트 페이지로 구성된(가상 페이지 0, 1, 2, 4)
작은 주소 공간의 예를 보여준다.

물론 실제 주소 공간은 훨씬 커서 32 비트의 경우 4GB, 64 비트의 경우에는 그보다
훨씬 크다.



여담으로, 64 비트 주소 공간은 상상하기 어려울 정도로 엄청나게 크다. 비유가 도움이 될 것이다. 32 비트 주소 공간을 테스트 코트라고 생각하면 64 비트 주소 공간은 유럽만하다.



물리 메모리는 그림과 같이, 고정 크기의 슬롯들로 구성된다. 이 경우 8 개의 페이지 프레임, 총 128 바이트의 비현실적으로 작은 물리 메모리이다.

그림에서 볼 수 있듯이, 가상 주소 공간의 페이지들은 물리 메모리 전체에 분산 배치되어 있다. 그림은 또한 운영체제가 자기자신을 위해서 물리 메모리의 일부를 사용하는 것도 보여준다.

앞으로 보겠지만, 페이지징은 이전 방식에 비해 많은 장점을 가지고 있다. 아마도 가장 중요한 개선은 유연성일 것이다.

페이징을 사용하면 프로세스의 주소 공간 사용 방식과는 상관없이 효율적으로 주소 공간 개념을 지원할 수 있다.

예를 들어, 힙과 스택이 어느 방향으로 커지는가, 어떻게 사용되는가에 대한 가정을 하지 않아도 된다.

또 다른 장점은 페이징이 제공하는 빈 공간 관리의 단순함이다.

예를 들어, 운영체제가 우리의 작은 64 바이트 주소 공간을 8 페이지 물리 메모리에 배치하기를 원한다고 할 때, 운영체제는 비어 있는 네 개의 페이지만 찾으면 된다. 아마 이를 위해 운영체제는 모든 비어 있는 페이지의 빈 공간 리스트를 유지하고 리스트의 첫 네 개 페이지를 선택할 것이다.

이 예에서, 운영체제는 주소 공간의 페이지 0 을 물리 프레임 3 에, 가상 페이지 1 을 물리 프레임 7, 페이지 2 를 프레임 5, 그리고 페이지 3 을 프레임 2 에 배치하였다. 페이지 프레임 1, 4, 6 은 현재 비어 있다.

주소 공간의 각 가상 페이지에 대한 물리 메모리 위치 기록을 위하여 운영체제는 프로세스마다 페이지 테이블(page table)이라는 자료 구조를 유지한다.

페이지 테이블의 주요 역할은 주소 공간의 가상 페이지 주소 변환(address translation) 정보를 저장하는 것이다.

각 페이지가 저장된 물리 메모리의 위치가 어디인지 알려준다.

위의 간단한 예제의 경우 페이지 테이블은 다음과 같이 4 개의 항목을 가지게 될 것이다.

(가상 페이지 0 → 물리 프레임 3), (VP1 → PF 7), (VP2 → PF5) 및 (VP3 → PF2).

페이지 테이블은 프로세스마다 존재한다는 사실을 숙지해야 한다.

위의 예에서 다른 프로세스를 실행해야 한다면 운영체제는 이 프로세스를 위한 다른 페이지 테이블이 필요하다.

새 프로세스의 가상 페이지는 다른 물리 페이지에 존재하기 때문이다.(공유 중인 페이지가 없다면)

이제 주소 변환 준비가 되었다.

작은 주소 공간(64 바이트)을 가진 프로세스가 다음 메모리 접근을 수행한다고 가정하자.

<code>movl <virtual address>, %eax</code>

구체적으로 주소 <virtual address>의 데이터를 eax 레지스터에 탑재하는 데에 집중하자.

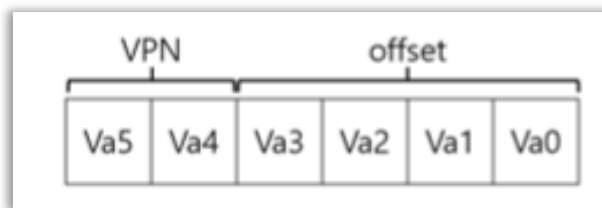
프로세스가 생성한 가상 주소의 변환을 위해 먼저 가상 주소를 가상 페이지 번호(virtual page number, VPN)와 페이지 내의 오프셋 2 개의 구성 요소로 분할한다.

이 예에서는 가상 주소의 크기가 64 바이트이기 때문에 가상 주소는 6 비트가 필요하다.($2^6 = 64$).

가상 주소를 개념적으로 다음 그림과 같이 생각할 수 있다.



이 그림에서 Va5 는 가상 주소의 최상위 비트이며, Va0 은 최하위 비트를 나타낸다. 우리는 페이지 크기(16 바이트)를 알고 있기 때문에, 다음과 같이 가상 주소를 나눌 수 있다.



페이지 크기는 64 바이트의 주소 공간에서 16 바이트이다.

따라서 4 페이지를 선택할 수 있어야 하고 따라서 주소의 최상위 2 비트가 그 역할을 한다.

우리는 2 비트 가상 페이지 번호(VPN)를 가지게 된다.

나머지 비트는 페이지 내에서 우리가 원하는 바이트의 위치를 나타낸다.

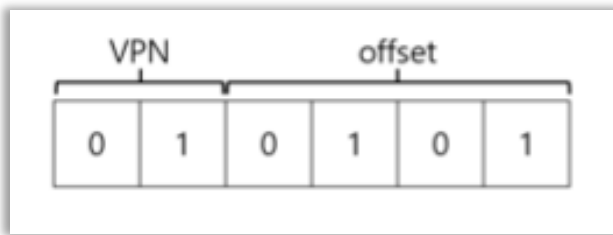
이것을 오프셋이라고 부른다.

프로세스가 가상 주소를 생성하면 운영체제와 하드웨어가 의미있는 물리 주소로 변환한다.

예를 들어, 위 탑재 명령어의 가상 주소가 21 이라고 하자.

```
movl 21, %eax
```

“21”을 이진 형식으로 변환하면서 “010101”을 얻고, 이 가상 주소를 검사하고 가상 페이지 번호와 오프셋으로 나눈다.



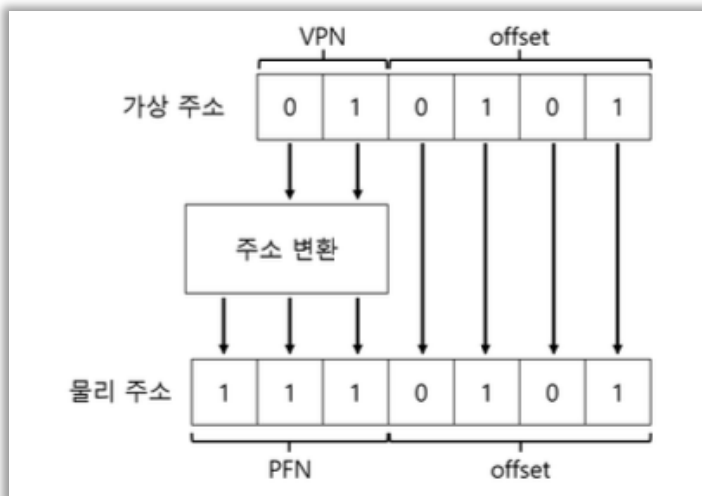
따라서 가상 주소 “21”은 가상 페이지 “01”(또는 1)의 5 번째(“0101”번째) 바이트이다.

이 가상 페이지 번호를 가지고 페이지 테이블의 인덱스로 사용하여 가상 페이지 1 이 어느 물리 프레임에 저장되어 있는지 찾을 수 있다.

위의 페이지 테이블에서 물리 프레임 번호(physical frame number, PFN) 혹은 물리 페이지 번호(physical page number, PPN)는 7(이진수 111)이다.

VPN 을 PFN 으로 교체하여 가상 주소를 변환할 수 있다.

그런 후에 물리 메모리에 탑재 명령어를 실행한다.



오프셋은 동일한다(즉, 변환되지 않는다)는 것에 주의하라.

오프셋은 페이지 내에서의 우리가 원하는 위치를 알려주기 때문이다.

최종적으로 계산된 물리 주소는 1110101(십진수 117)이며, 이곳이 탑재할 데이터가 저장된 정확한 위치이다.

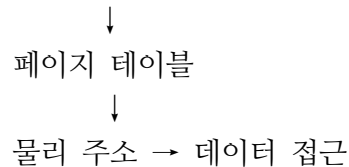
[2] 페이지 테이블은 어디에 저장되는가

페이지 테이블은 매우 커질 수 있다.

우리가 이전에 논의했던 작은 세그먼트 테이블이나 베이스-바운드 쌍에 비해서 말이다.

예를 들어, 4KB 크기의 페이지를 가지는 전형적인 32 비트 주소 공간을 상상해보자.

가상 주소(32bit) → [페이지 번호(20bit) | offset(12bit)]



이 가상 주소는 20 비트 VPN 과 12 비트 오프셋으로 구성된다.

1KB 페이지 크기를 위해 10 비트가 필요하며, 4KB 를 위해서는 두 비트만 추가하면 된다.

20 비트 VPN 은 운영체제가 각 프로세스를 위해 관리해야 하는 변환의 개수가 2^{20} 이라는 것을 의미한다.(어림잡아 백만)

물리 주소로의 변환 정보와 다른 필요한 정보를 저장하기 위하여 페이지 테이블 항목(page table entry, PTE) 마다 4 바이트가 필요하다고 가정하면, 각 페이지 테이블을 저장하기 위하여 4MB 의 메모리가 필요하게 된다.

이것은 꽤 커다란 크기이다.

프로세스 100 개가 실행 중이라고 가정하자.

주소 변환을 위해서 운영체제가 400MB 의 메모리를 필요로 하는 것을 의미한다.

컴퓨터가 Giga byte 단위의 메모리를 가지고 있는 현재라도 변환을 위해서 이런 큰 청크를 사용한다는 것은 매우 비정상적이다.

64 비트 주소 공간을 위한 페이지 테이블의 크기가 얼마나 클지에 대해서는 생각하기조차 싫다.

페이지 테이블이 매우 크기 때문에 현재 실행 중인 프로세스의 페이지 테이블을 저장할 수 있는 회로를 MMU 안에 유지하지 않을 것이다.

대신 각 프로세스의 페이지 테이블을 메모리에 저장한다.

당분간 페이지 테이블은 운영체제가 관리하는 물리 메모리에 상주한다고 가정하자.

나중에 운영체제 메모리 자체의 많은 부분이 가상화될 수 있다는 것을 알게 될 것이다.

그러나 현재로서는 매우 혼란스럽기 때문에 일단 무시한다. 운영체제 메모리 영역(PFN 0)에 페이지 테이블이 존재한다.

[3] 페이지 테이블에는 실제 무엇이 있는가

페이지 테이블 구성에 대해 살펴보자.

페이지 테이블은 가상 주소(또는 실제로는 가상 페이지 번호)를 물리 주소(물리 프레임 번호)로 매핑하는데 사용되는 자료 구조이다.

임의의 자료 구조도 사용 가능하다.

가장 간단한 형태는 선형 페이지 테이블(linear page table)이다. 단순한 배열이다.

운영체제는 원하는 물리 프레임 번호(PFN)를 찾기 위하여 가상 페이지 번호(VPN)로 배열의 항목에 접근하고 그 항목의 페이지 테이블 항목(PTE)을 검색한다.

지금은 이 간단한 선형 구조를 사용한다고 가정할 것이다.

이후의 장에서 페이징과 관련된 문제를 해결하기 위한 고급 자료 구조를 사용할 것이다.

0	페이지 테이블: 3 7 5 2	물리 메모리의 페이지 프레임 0
16	(미사용)	페이지 프레임 1
32	AS의 페이지 3	페이지 프레임 2
48	AS의 페이지 0	페이지 프레임 3
64	(미사용)	페이지 프레임 4
80	AS의 페이지 2	페이지 프레임 5
96	(미사용)	페이지 프레임 6
112	AS의 페이지 1	페이지 프레임 7
128		

각 PTE 에는 심도있는 이해가 필요한 비트들이 존재한다.

[Valid bit]

Valid bit 는 특정 변환의 유효 여부를 나타내기 위하여 포함된다.

예를 들어, 프로그램이 실행을 시작할 때 코드와 힙이 주소 공간의 한쪽에 있고 반대쪽은 스택이 차지하고 있을 것이다.

그 사이의 모든 미사용 공간은 무효(invalid)로 표시되고, 프로세스가 그런 메모리를 접근하려고 하면 운영체제에 트랩을 발생시킨다.

운영체제는 그 프로세스를 종료시킬 확률이 높다.

Valid bit 는 할당되지 않은 주소 공간을 표현하기 위해 반드시 필요하다.

주소 공간의 미사용 페이지를 모두 표시함으로써 이러한 페이지들에게 물리 프레임을 할당할 필요를 없애 대량의 메모리를 절약한다.

[Protection bit]

페이지가 읽을 수 있는지, 쓸 수 있는지, 또는 실행될 수 있는지를 표시하는 protection bit 가 있다.

Protection bit 가 허용하지 않는 방식으로 페이지에 접근하려고 하면 운영체제에 트랩을 생성한다.

중요하지만 당장은 다루지 않는 몇 가지 다른 비트가 있다.

[Present bit]

Present bit 는 이 페이지가 물리 메모리에 있는지 혹은 디스크에 있는지(즉, 스왑 아웃되었는지) 가리킨다.

이런 기본적인 기법들은 나중에 물리 메모리보다 더 큰 주소 공간을 지원하기 위하여 주소 공간의 일부를 스왑하는 바업에 대해 배울 때 좀 더 자세히 이해할 것이다. 스와핑은 운영체제가 드물게 사용되는 페이지를 디스크로 이동시켜 물리 메모리를 비울 수 있게 한다.

[dirty bit]

dirty bit 또한 일반적인데, 메모리에 반입된 후 페이지가 변경되었는지 여부를 나타낸다.

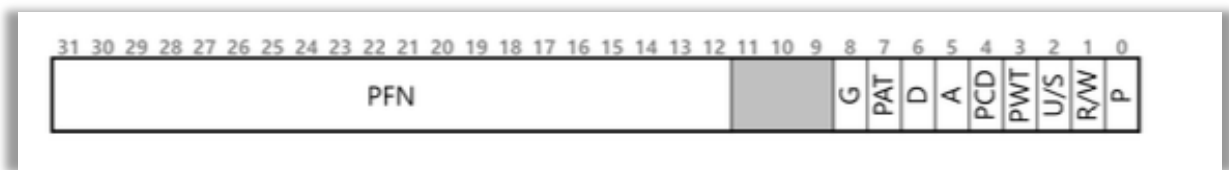
[reference bit(accessed bit)]

reference bit(accessed bit)는 때때로 페이지가 접근되었는지를 추적하기 위해 사용된다.

또한, 어떤 페이지가 인기가 있는지 결정하여 메모리에 유지되어야 하는 페이지를 결정하는 데에도 유용하다.

이 정보는 페이지 교체에 매우 중요하다.

페이지 교체에 대해서는 이후의 장에서 매우 자세하게 다룬다.



위 그림은 x86 아키텍처의 페이지 테이블 항목을 보여준다.

이 항목은 다음과 같은 비트들을 가진다.

- Present bit(P): 메모리에 있는지
- R/W(Read/Write): 쓰기 가능
 - 0: 읽기만
 - 1: 쓰기 가능
- U/S(User/Supervisor): 누가 접근 가능
 - 0: 커널만
 - 1: 유저도 가능
- A(Accessed bit): 사용된 적 있는지
- D(Dirty bit): 수정된 적 있는지

[4] 페이지: 너무 느림

페이지 테이블의 크기가 메모리 상에서 매우 크게 증가할 수 있다.

페이지 테이블로 인해 처리 속도가 저하될 수 있다. 간단한 명령어를 예로 들어 보자.

```
movl 21, %eax
```

주소 21 에 대한 참조만 고려하고 명령어 반입에 대해서는 고려하지 않기로 하자.

이 예제에서 하드웨어가 주소 변환을 담당한다고 가정한다.

원하는 데이터를 가져오기 위해, 먼저 시스템은 가상 주소(21)를 정확히 물리적 주소(117)로 변환해야 한다.

주소 117 에서 데이터를 반입하기 전에 시스템은 프로세스의 페이지 테이블에서 적절한 페이지 테이블 항목을 가져와야 하고, 변환을 수행한 후, 물리 메모리에서 데이터를 탑재한다.

이렇게 하기 위해서, 하드웨어는 현재 실행 중인 프로세스의 페이지 테이블의 위치를 알아야 한다.

당분간 하나의 페이지 테이블 베이스 레지스터(page table base register)가 페이지 테이블의 시작 주소(물리 주소)를 저장한다고 가정한다.

원하는 PTE 의 위치를 찾기 위해 하드웨어는 다음과 같은 연산을 수행한다.

[4-1] 주어진 값 정리

- 가상 주소 = 21 → 010101(6 비트)
- 페이지 크기 = 16B → offset = 4 비트
- [VPN (2 비트) | offset (4 비트)]

[4-2] VPN 구하는 과정

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
```

(1) VPN_MASK 의미

```
VPN_MASK = 110000
```

- VPN 은 앞 2 비트만 필요
- 그래서 앞만 남기고 나머지 제거

(2) 실제 적용

```
VirtaulAddress = 010101
```

```
VPN_MASK      = 110000
```

AND 연산

```
010101
110000
-----
010000
```

(3) SHIFT

SHIFT = 4(offset 비트 수)

오른쪽으로 4 칸 이동

010000 >> 4 = 000001 = 1

결과

VPN = 1

[4-3] 페이지 테이블 접근

PTEAddr = PageTableBaseRegister + (VPN + sizeof(PTE))

- 페이지 테이블에서 1 번째 칸 찾기

결과

PFN = 7(예제 값)

[4-4] offset 구하기

offset = VirtualAddress & OFFSET_MASK

OFFSET_MASK

00001111(하위 4 비트)

계산

```
010101
00001111
-----
00000101 = 5
```

결과

offset = 5

[4-5] 최종 물리 주소 만들기

$\text{PhysAddr} = (\text{PFN} \ll \text{SHIFT}) \mid \text{offset}$

계산

$\text{PFN} = 7 \rightarrow 111$

왼쪽 쉬프트:

$111 \ll 4 = 1110000$

이제 offset 붙이기:

```
1110000
0000101
-----
1110101
```

최종 결과

$1110101 = 117$

[4-6] 전체 흐름

가상주소 $\rightarrow$ VPN 추출 $\rightarrow$ 페이지테이블 조회 $\rightarrow$ PFN 얻기
 $\rightarrow$ PFN + offset 합쳐서 물리주소 생성

1. MASK $\rightarrow$ 필요한 비트만 뽑기
2. SHIFT $\rightarrow$ 위치 맞추기
3. $\text{PFN} \ll \text{SHIFT} \rightarrow$ 페이지 시작 위치 만들기

```

// 가상 주소에서 VPN 추출
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
// 페이지 테이블 항목(PTE)의 주소 형성
PTEAddr = PTBR + (VPN + sizeof(PTE))
// PTE 반입
PTE = AccessMemory(PTEAddr)
//프로세스가 페이지를 접근할 수 있는지 확인)
if(PTE.Valid == False)
    RaiseException(SEGMENTATION_FAULT)
else if(CanAccess(PTE.ProtectBits) == False)
    RaiseException(PROTECTION_FAULT)
else
    // 접근 가능하면 물리 주소 만들고 값 가져오기
    offset = VirtualAddress & OFFSET_MASK
    PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
    Register = AccessMemory(PhysAddr)

```

[5] 메모리 트레이스

마치기 전에, 간단한 메모리 액세스 예를 통해 페이징을 사용했을 때 발생하는 모든 메모리 접근을 살펴보자.

우리가 관심 있는 코드는(파일 array.c 의 C 언어로 된)다음과 같다.

```

int array[1000];
...
for(i = 0; i < 1000; i++)
    array[i] = 0;

```

우리는 array.c 를 컴파일하고 다음과 같이 실행한다.

```

clang -g array.c -o array
./array

```

물론, 이 코드(단순히 배열을 초기화한다)가 어떤 메모리 접근을 생성할지에 대해서 진짜로 이해하려면 우리는 몇 가지 사실을 더 알아야 한다.

먼저, 루프 안에서 배열을 초기화하기 위해 어떤 어셈블리 명령어를 사용하는지 보기 위하여 결과 이진 파일을 디스어셈블 해야만 한다.(Linux: objdump, Mac: otool)

```
0x1024 movl $0x0, (%edi, %eax, 4)
0x1028 incl %eax
0x102c cmpl $0x03e8, %eax
0x1030 jne 0x1024
```

x86 을 조금 알고 있다면, 위 코드는 이해하기 아주 쉽다.

첫 번째 명령어는 값 0(0x0)을 가상 메모리 주소로 옮긴다.

0 값이 저장될 가상 메모리 주소는 %edi 의 값을 %eax 의 4 배에다 더해서 계산된다.

%edi 는 배열의 시작 주소를 저장하고, %eax 는 배열 인덱스(i)를 저장한다.

배열이 정수 배열이고 정수의 크기는 4 바이트이기 때문에 4 를 곱한다.

즉, eax 를 인덱스로 해서 edi 배열에 접근하고, 거기에 0 을 넣는다.

두 번째 명령어는 %eax 에 저장된 배열 인덱스를 증가시키고,

세 번째 명령어는 %eax 의 값과 16 진수 0x03e8 또는 십진수 1000 을 비교한다.

비교 결과 아직 두 값이 같지 않다면(jne 명령어가 검사하는 것),

네 번째 명령어는 루프의상단으로 다시 분기한다.

이 명령어 시퀀스(가상 및 물리 수준 모두에서) 어떤 메모리 접근을 생성하는지 이해하기 위해서, 코드와 배열의 가상 메모리의 주소와 페이지 테이블의 위치에 대해서 몇 가지 가정을 해야 한다.

[5-1] 예제; 페이징

(1) 기본 조건

가상 주소 공간: 64KB

페이지 크기: 1KB

총 페이지 수 = 64 개(VPN 0~63)

(2) 페이지 테이블

형태: 선형(배열)

위치: 물리 메모리 1024 번지

역할: VPN → PFN 변환

* 프로세스마다 존재

(3) 주요 메모리 배치

코드

가상 주소 1024 → VPN 1
→ PFN 4

배열

크기: 4000B (int 1000 개)
주소: 40000 ~ 44000

- 페이지 범위: VPN 39~42
- 매핑
 - VPN 39 → PFN 7
 - VPN 40 → PFN 8
 - VPN 41 → PFN 9
 - VPN 42 → PFN 10

(4) 메모리 접근 구조

모든 접근 공통

1. 페이지 테이블 접근
 2. 실제 메모리 접근
- 항상 2 번 접근 발생

(5) 루프 1 번 실행 시

1. 명령어 fetch(4 개)

각각:
페이지 테이블 + 명령어 읽기 = 2 번
→ 총 8 번

2. 데이터 접근(mov)

페이지 테이블 + 실제 배열 접근 = 2 번

총합

루프 1 회 = 10 번 메모리 접근

느린 이유는 페이징에서 가상 주소를 물리 주소로 변환하기 위해
매 메모리 접근마다 페이지 테이블을 먼저 참조해야 하므로 추가적인 메모리 접근이
발생하여 성능이 저하된다.

[1-2] 가상화: 더 빠른 변환(TLB)

핵심 질문: 주소 변환 속도를 어떻게 향상할까

주소 변환을 어떻게 빨리할 수 있을까?

페이징에서 발생하는 추가 메모리 참조를 어떻게 피할 수 있을까?

어떤 하드웨어가 추가로 필요한지?

운영체제가 어떤식으로 개입해야 할까?

페이징은 상당한 성능 저하를 가져올 수 있다.

페이징은 프로세스 주소 공간을 작은 고정된 크기(즉, 페이지)로 나누고 각 페이지의 실제 위치(매핑 정보, mapping information)를 메모리에 저장된다.

매핑 정보를 저장하는 자료 구조를 페이지 테이블이라 한다.

매핑 정보 저장을 위해 큰 메모리 공간이 요구된다.

무엇보다 중요한 것은 가상 주소에서 물리 주소로의 주소 변환을 위해 메모리에 존재하는 매핑 정보를 읽어야 한다는 사실이다.

페이지 테이블 접근을 위한 메모리 읽기 작업은 엄청난 성능 저하를 유발한다.

모든 load/store 명령어 실행이 추가적인 메모리 읽기를 수반하는 상황을 가정해보라. 정말 머리 아픈 일이 아닐 수 없다.

운영체제의 실행 속도를 개선하려면, 도움이 필요하다.

대부분의 경우 하드웨어로부터 도움을 받는다.

하드웨어는 운영체제의 오랜 친구!

주소 변환을 빠르게 하기 위해서 우리는 변환-색인 버퍼(translation-lookaside buffer) 또는 TLB 라고 부르는 것을 도입한다.

TLB 는 칩의 메모리 관리부(memory-management unit, MMU)의 일부이다.

자주 참조되는 가상 주소-실주소 변환 정보를 저장하는 하드웨어 캐시이다.

주소-변환 캐시(address-translation cache)가 더 적합한 명칭이다.

가상 메모리 참조 시, 하드웨어는 먼저 TLB 에 원하는 변환 정보가 있는지를 확인한다.

만약 있다면 페이지 테이블(모든 변환 정보를 갖고 있음)을 통하지 않고 변환을 (빠르게) 수행한다.

실질적으로 TLB 는 페이징 성능을 엄청나게 향상 시킨다.

TLB 를 도입함으로써, 페이징이 “사용 가능”한 가상 메모리 기법이 된다.

[1] TLB 의 기본 알고리즘

```
1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True) // TLB 히트
4     if (CanAccess(TlbEntry.ProtectBits) == True)
5         Offset = VirtualAddress & OFFSET_MASK
6         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7         AccessMemory(PhysAddr)
8     else
9         RaiseException(PROTECTION_FAULT)
10 else // TLB 미스
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```

그림은 가상 주소 변환이 이루어지는 과정을 대략적으로 나타내고 있다. 그림에서는 주소 변환부가 단순한 선형 페이지 테이블(배열)과 하드웨어로 관리되는 TLB 로 구성되어 있다.

하드웨어 부분의 알고리즘은 다음과 같이 동작한다.

먼저, 가상 주소에서 가상 페이지 번호(VPN)을 추출한 후, 해당 VPN 의 TLB 존재 여부를 검사한다. (라인 2)

만약 존재하면 TLB 히트이고 TLB 가 변환 값을 갖고 있다는 것을 뜻한다. 성공 이제 해당 TLB 항목에서 페이지 프레임 번호(PFN)를 추출할 수 있다. 해당 페이지에 대한 접근 권한 검사가 성공하면(라인 4), 그 정보를 원래 가상 주소의 오프셋과 합쳐서 원하는 물리 주소(PA)를 구성하고, 메모리에 접근할 수 있다.(라인 5-7)

TLB 에 변환 정보가 존재하지 않는다면 (TLB 미스) 할 일이 많다.

이 예제에서는 하드웨어가 변환 정보를 찾기 위해서 페이지 테이블에 접근하며(라인 11-12),

프로세스가 생성한 가상 메모리 참조가 유효하고 접근이 가능하다면(라인 13-15), 해당 변환 정보를 TLB 로 읽어들인다(라인 18), 매우 시간이 많이 소요되는 작업이다.

페이지 테이블 접근을 위한 메모리 참조 때문이다.(라인 12)

TLB 가 갱신되면 하드웨어는 명령어를 재실행한다.

이번에는 TLB 에 변환 정보가 존재하므로, 메모리 참조가 빠르게 처리된다.

모든 캐시의 설계 철학처럼, TLB 역시 “주소 변환 정보가 대부분의 경우 캐시에 있다”(캐시 히트)라는 가정을 전제로 만들어졌다.

TLB는 프로세싱 코어와 가까운 곳에 위치하고 있고, 매우 빠른 하드웨어로 구성되기 때문에, 주소 변환 작업은 그다지 부담스러운 작업이 아니다.

미스가 발생하는 경우, 페이징 비용이 커진다.

페이지 테이블을 접근하여 변환 정보를 찾아야 한다.

메모리 참조(또는 페이지 테이블이 복잡하다면 더 많이)가 추가된다.

이 상황이 자주 일어나면 프로그램은 상당히 느려지게 된다.

메모리 접근 연산은 다른 CPU 연산(예) 더하기, 곱하기 등)에 비해 매우 시간이 오래 걸린다.

TLB 미스가 많이 발생할수록 메모리 접근 횟수가 많아진다.

TLB 미스가 발생하는 경우를 최대한 피해야 한다.

[2] 예제: TLB와 배열 접근 예제

[2-1] 개요

TLB는 가상 주소를 물리 주소로 변환할 때, 페이지 테이블을 참조하는 비용을 줄여주는 캐시 역할을 한다.

TLB가 잘 활용되면 메모리 접근 성능을 크게 개선할 수 있다.

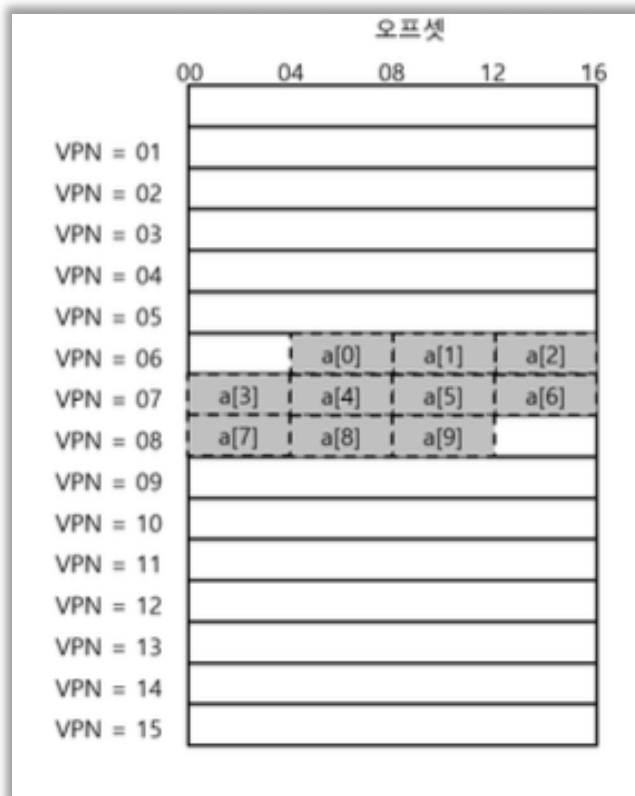
이번 예제에서는 작은 가상 주소 공간에서 배열을 접근하며 TLB의 동작성과 성능 효과를 살펴본다.

[2-2] 실습 예제 설정

- 배열: 정수형 10개(a[0] ~ a[9])
- 각 정수 크기: 4바이트
- 가상 주소 공간: 8비트
- 페이지 크기: 16바이트
- 가상 주소 구성
 - VPN: 상위 4비트 → 가상 페이지 번호
 - Offset: 하위 4비트 → 페이지 내 오프셋

배열 배치

- a[0]~a[2]: VPN=6, 페이지 내 연속
- a[3]~a[6]: VPN=7, 페이지 내 연속
- a[7]~a[9]: VPN=8, 페이지 내 연속



[2-3] 배열 합 계산 코드

```
int sum = 0;
for (int i = 0; i < 10; i++) {
    sum += a[i];
}
```

단, TLB 동작 분석에서는 i, sum, 루프 명령어에 대한 메모리 접근은 무시하고, 배열 원소 접근만 고려한다.

[2-4] TLB 동작 과정

배열 원소	가상 주소	VPN	TLB 동작	설명
a[0]	100	6	Miss	TLB 초기화 상태, 페이지 테이블 참조 후 TLB 갱신
a[1]	104	6	Hit	같은 페이지 내 접근, TLB 활용
a[2]	108	6	Hit	TLB 활용
a[3]	112	7	Miss	새로운 페이지, TLB 갱신 필요
a[4]	116	7	Hit	같은 페이지 내 접근
a[5]	120	7	Hit	TLB 활용
a[6]	124	7	Hit	TLB 활용
a[7]	128	8	Miss	새로운 페이지, TLB 갱신 필요
a[8]	132	8	Hit	TLB 활용
a[9]	136	8	Hit	TLB 활용

- TLB 히트율: $7/10 = 70\%$

- 배열 접근 시 공간 지역성 덕분에 페이지 내 첫 접근만 미스가 발생하고, 이후 접근은 히트가 된다.

[2-5] 성능 개선 요인

1. 페이지 크기

- 페이지 크기가 커지면 한 페이지 내 더 많은 데이터가 들어가므로 TLB 미스가 감소

- 예: 페이지 크기를 32 바이트로 늘리면 TLB 미스가 더 줄어든다.

2. 시간 지역성

- 한 번 참조한 메모리 영역이 짧은 시간 내 재참조될 때 TLB 히트율 증가

- 예: 루프 종료 후 배열을 다시 사용하면 모든 주소 변환 정보가 TLB에 남아 히트율 100% 달성 가능

3. 공간 지역성

- 연속된 메모리 접근이 TLB 효율 향상에 도움

- 배열, 구조체 등 연속 메모리 구조를 사용할 때 성능 개선 효과가 큼

[2-6] 요약

- TLB는 주소 변환 캐시로, 페이지 테이블 참조 비용을 줄여 메모리 접근 성능을 높임
- 배열 접근 예제에서 첫 번째 페이지 접근은 TLB 미스, 이후 같은 페이지는 히트
- TLB 효율은 페이지 크기, 공간 및 시간 지역성에 크게 의존
- 실제 프로그램은 대부분 공간/시간 지역성을 갖고 있어 TLB 활용 효과가 큼

[3] TLB 미스처리: 하드웨어 vs 소프트웨어

[3-1] 개요

TLB 미스가 발생하면, 가상 주소를 물리 주소로 변환하지 못한 상태가 된다.
이때 누가 미스를 처리하는가에 따라 처리 방식이 달라진다.

두 가지 접근이 존재한다.

1. 하드웨어 관리 TLB
2. 소프트웨어 관리 TLB

[3-2] 하드웨어 관리 TLB(CISC 계열)

- 대표 사례: 인텔 x86 CPU
- 과거 CISC(Complex Instruction Set Computer) 구조에서 등장
- 하드웨어가 페이지 테이블 구조와 위치를 알고 있어야 TLB 미스를 처리할 수 있음

미스 처리 과정

1. TLB 미스 발생
2. 하드웨어가 페이지 테이블에서 해당 페이지 엔트리 조회
3. 변환 정보 추출 → TLB 갱신
4. TLB 미스가 발생한 명령어 재실행
 - x86 CPU는 멀티 레벨 페이지 테이블을 사용
 - CR3 레지스터가 페이지 테이블의 시작 주소를 저장

특징: 미스를 처리하는 모든과정이 하드웨어 내부에서 자동으로 이루어짐

[3-3] 소프트웨어 관리 TLB(RISC 계열)

- 대표 사례: MIPS R10K, SPARC v9
- RISC(Reduced Instruction Set Computer) 구조에서 등장
- 하드웨어는 TLB 미스 발생 시 예외 신호를 발생시킴
- 운영체제가 TLB 미스를 처리하는 방식

미스 처리 과정

1. TLB 에서 주소 변환 실패 → 하드웨어가 예외 발생
2. CPU 실행 중지 → 커널 모드 전환
 - 커널 주소 공간 접근 가능
 - 특권 레벨(privilege level) 상향 조정
3. 트랩 핸들러 실행
 - 운영체제 코드가 TLB 미스를 처리
 - 페이지 테이블 검색 → 변환 정보 추출
 - 특권 명령어로 TLB 갱신
4. 트랩 핸들러 종료 → 명령어 재실행
 - 이번에는 TLB 히트 발생

주의 사항

- 트랩 핸들러와 일반 시스템 콜 트랩과 다름
 - TLB 미스 처리 후 명령어는 재실행
 - 시스템 콜 처리 후 명령어는 다음 명령어 실행
- TLB 미스 핸들러가 무한 반복되지 않도록 조치 필요
 - 예: 핸들러를 물리 주소 기반으로 배치
 - 또는 TLB 일부를 핸들러 코드용으로 영구 할당 → wired 변환

장점:

- 페이지 테이블 구조를 유연하게 변경 가능
- 하드웨어가 단순 → 설계 및 구현 간단

[3-4] 정리

구분	처리 주체	특징	대표 사례
하드웨어 관리	CPU	TLB 미스를 하드웨어가 직접 처리	x86
소프트웨어 관리	OS	TLB 미스를 운영체제 트랩 핸들러가 처리	MIPS, SPARC

- 핵심 차이: 미스 발생 시 누가 주소 변환과 TLB 갱신을 담당하는지
- 하드웨어 관리: 자동, CPU 내부 처리
- 소프트웨어 관리: 예외 → OS 트랩 핸들러 → 재실행

결론: 소프트웨어 관리 TLB는 유연성과 단순함이 장점이며, 하드웨어 변경없이 페이지 테이블 구조를 자유롭게 설계 가능

[4] 하드웨어 TLB 구성과 연관 방식

[4-1] TLB 개요

- TLB는 가상 주소(VPN) → 물리 주소(PFN) 변환 정보를 저장하는 캐시
- 일반적으로 32~128개 엔트리를 가지며, CPU에서 주소 변환 시 매우 빠르게 접근을 가능하게 한다.
- TLB 항목 구성:

필드	설명
VPN (Virtual Page Number)	가상 페이지 번호
PFN (Physical Frame Number)	물리 페이지 번호
Valid bit	항목이 유효한지 여부
Protection bit	페이지 접근 권한 (읽기, 쓰기, 실행 등)
Dirty bit	페이지가 수정되었는지 여부
Address-space ID	프로세스 주소 공간 구분

[4-2] 연관 방식

완전 연관(fully associative)

- 모든 TLB 엔트리를 대상으로 병렬 검색 가능
- 변환 정보가 TLB 내 어디에나 위치할 수 있음
- 장점: TLB 활용 효율이 가장 높음(적절한 엔트리만 있으면 히트율 극대화)
- 단점: 하드웨어 설계가 복잡, 병렬 비교 회로 필요

예시: 64개 엔트리 TLB → CPU는 64개의 VPN을 동시에 비교하여 히트 여부 결정

부분 연관(set-associative)

- TLB 를 여러 세트(set)로 나누고, 각 세트내에서만 완전 연관 검색
- 예: 64-entry, 4-way set-associative → 16 세트 x 4-way
- 장점: 설계 단순함, 검색 속도 유지
- 단점: 완전 연관보다 히트율 약가 ↓ 감소 가능

직접 매핑(direct-mapped)

- 각 VPN 이 딱 한 위치에만 매핑됨
- 장점: 구현 단순, 검색 속도 매우 빠름
- 단점: 충돌 시 히트율 감소 가능(같은 세트에 여러 VPN 몰림)

[5] TLB 와 문맥 교환 문제

[5-1] 문제의 발생

- TLB 는 가상 주소 → 물리 주소 변환 정보를 캐시한다.
- TLB 의 변환 정보는 특정 프로세스에만 유효하다.

예시

- 프로세스 P1: VPN 10 → PFN 100
- 프로세스 P2: VPN 10 → PFN 170

문맥 교환 후 P2 가 실행되면, TLB 에 남아 있는 P1 의 변환 정보를 그대로 사용하면 잘못된 접근이 발생한다.

즉, TLB 는 멀티 프로세스 환경에서 단순히 공유될 수 없다.

[5-2] 전통적 해결책: TLB 플러시(TLB Flush)

- 문맥 교환 시 TLB 의 모든 valid bit 를 0 으로 초기화
- 하드웨어 또는 소프트웨어 기반 명령어로 수행 가능
 - 하드웨어 관리 TLB: 페이지 테이블 베이스 레지스터(PTBR) 변경 시 자동 초기화
 - 소프트웨어 관리 TLB: 특권 명령어를 사용하여 TLB 비우기
- 장점: 이전 프로세스의 변환 정보가 새로운 프로세스에 영향을 주지 않음
- 단점: 문맥 교환 후 새로운 프로세스 접근마다 TLB 미스 발생 → 성능 부담

[5-3] 개선된 해결책: ASID(Address Space Identifier)

- 하드웨어 기능 추가
- TLB 엔트리에 ASID 필드 추가 → 프로세스별 TLB 정보 구분
- ASID 는 프로세스 식별자(PID)보다 작은 비트 수를 사용
 - 예: PID 32 비트, ASID 8 비트
- 문맥 교환 시: 운영체제는 새로운 ASID 를 CPU 레지스터에 설정
- 이 방식으로 TLB 를 플러시하지 않고도 여러 프로세스의 변환 정보를 공존 가능

ASID 를 활용한 TLB 예시

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
10	170	1	rwX	2

- 현재 실행 프로세스의 ASID 와 일치하는 TLB 항목만 사용됨
- 다른 프로세스 항목은 무시 → 안전하게 다중 프로세스 지원 가능

[5-4] 공유 페이지 처리

- 서로 다른 프로세스가 같은 물리 페이지를 공유할 수 있음
 - 예: 코드 페이지, 라이브러리
- TLB 예시

VPN	PFN	valid	prot	ASID
10	101	1	r-x	1
50	101	1	r-x	2

- P1 과 P2 가 동일한 물리 페이지(PFN 101)를 사용하더라도, 각자 다른 VPN 과 ASID 를 가짐
- 장점: 물리 페이지 절약, 메모리 부하 감소

[5-5] 요약

- 문맥 교환 시 TLB 문제: 이전 프로세스 변환 정보가 새로운 프로세스에 영향을 줄 수 있음
 - 해결책 1: TLB 플러시 → 간단하지만 성능 부담
 - 해결책 2: ASID → 성능 유지, 안전하게 다중 프로세스 지원
 - 공유페이지도 ASID 와 VPN 조합으로 TLB 에서 안전하게 처리 가능
- 핵심: ASID 도입으로 문맥 교환 시 TLB 히트율을 유지하면서 안전성을 확보

[6] 이슈: 교체 정책

모든 캐시가 그러하듯이 TLB 에서도 캐시 교체 정책이 매우 중요하다.

TLB 에 새로운 항목을 탑재할 때, 현재 존재하는 항목 중 하나를 교체 대상으로 선정해야 한다.

어느 것을 선택해야 할까?

이 내용은 디스크와 메모리 간의 페이지 스와핑 부분에서 상세히 다루도록 하고, 여기서는 몇 개의 일반적인 정책들만 요약하도록 하겠다.

한 가지 흔한 방법은 가장 오래전부터 사용되었던 최저 사용 빈도(Least-Recently-used, LRU) 항목을 교체하는 것이다.

LRU 는 메모리 참조 패턴에서의 지역성을 최대한 활용하는 것이 목적이다.

사용되지 않은지 오래된 항목일수록, 앞으로도 사용될 가능성이 작으며, 교체 대상으로 적합하다는 가정에 근거한다.

또 다른 일반적인 방법은 랜덤(Random) 정책이다.

랜덤 정책에서는 교체 대상을 무작위로 정한다.

랜덤 교체 정책은 때때로 잘못된 결정을 내릴 수 있다.

하지만, 랜덤 교체 정책은 구현이 간단하고 예상치 못한 예외 상황의 발생을 피할 수 있다는 장점이 있다.

예를 들어, LRU 와 같은 “합리적인” 정책은 n 개의 변환 정보를 저장할 수 있는 TLB 가 $n + 1$ 페이지들에 대해서 반복문을 수행하는 프로그램에서 사용될 경우 최악의 TLB 미스를 생성하게 된다.

[7] 실제 TLB

마지막으로 실제 TLB가 어떻게 생겼는지 간략히 살펴보자.

MIPS R4000을 예로 들겠다.

MIPS R4000은 소프트웨어로 관리되는 TLB를 사용한다.

비교적 최근 시스템이다.

MIPS의 간략화된 TLB 항목은 그림과 같다.



MIPS R4000은 32비트 주소 공간에 4KB 페이지를 지원한다.

일반적인 가상 주소 공간을 사용하기 때문에 VPN은 20비트 그리고 오프셋으로 12비트를 갖을 것이라 예측할 수 있지만 아니다.

TLB에서 보는 바와 같이 19비트가 VPN에 할당되어 있다.

전체 주소 공간의 절반만 사용자 주소 공간으로 할당되어 있기(나머지 반은 커널이 사용)때문에, VPN에 19비트가 할당되어 있다.

물리 프레임 번호(PFN)로 24비트가 할당되어 있다.

64GB의 (물리) 주 메모리(2^{24} 개의 4KB 페이지들) 지원이 가능하다.

MIPS의 TLB에는 몇 가지 중요한 비트들이 더 있다.

전역(global) 비트 (G)이다.

이 비트는 프로세스들 간에 공유되는 페이지들을 위해 사용된다.

전역 비트가 설정되어 있으면 ASID는 무시된다.

8비트 길이의 ASID 필드가 있다.

운영체제는 이 부분을 보고 주소 공간들을 서로 구분한다.

여기서 재미있는 질문을 할 수 있다.

만일 $256(2^8)$ 개 이상의 프로세스들이 동시에 실행된다면?

마지막으로 3비트 길이의 일관성(coherence, C) 비트를 볼 수 있다.

이 비트를 보고 페이지가 하드웨어에 어떻게 캐시되어 있는지 판별한다.

더티(dirty)비트는 페이지가 갱신되면 세팅되며,

유효(valid) 비트는 항목에 유효한 변환 정보가 존재하는지를 나타낸다.

그리고 페이지 마스크(page mask) 필드도 있는데 (표현은 안되었음), 여러 개의 페이지 크기를 지원할 때 사용된다.

대용량 페이지 크기의 유용함에 대해서는 곧 배울 것이다.

마지막으로 회색 처리된 64 번째 비트는 사용하지 않는다.

MIPS 의 TLB 들은 일반적으로 32 개 또는 64 개의 항목으로 구성된다.

대부분은 사용자 프로세스들이 사용한다.

몇 개는 운영체제를 위해서 예약이 되어 있다.

운영체제에 의해서 연결된(wired) 레지스터가 설정이 될 수도 있으며, 운영체제는 몇 개의 TLB 를 예약할지를 하드웨어에게 알린다.

운영체제는 예약된 매핑 영역을 TLB 미스가 발생해서는 안되는(예: TLB 미스핸들러) 코드와 데이터를 위해서 사용한다.

MIPS 의 TLB 는 소프트웨어가 관리하기 때문에 TLB 를 갱신하기 위한 명령어가 필요하다.

MIPS 는 네 개의 명령어를 제공한다.

TLBP 는 어떤 특정 변환 정보가 존재하는지 탐색하는 데 사용이 되며 TLBR 은 TLB 항목의 내용을 레지스터로 읽어 드리는데 사용된다.

TLBWT 는 특정 TLB 항목을 교체하며 TLBWR 은 임의의 TLB 항목을 교체한다.

운영체제는 이 명령어들을 사용하여 TLB 의 내용을 관리한다.

이 명령어들은 특별한 실행권한을 갖고 있어야 한다.

만약 사용자 프로세스가 TLB 내용을 변경할 수 있다면 어떻게 될 지 상상해보라.

[1-3] 가상화: 페이징_더 작은 테이블

핵심 질문: 페이지 테이블을 어떻게 더 작게 만들까

단순한 배열 기반의 페이지 테이블은 (흔히 선형 페이지 테이블이라고 불림) 크기가 크면 일반적인 시스템에서 메모리를 과도하게 차지한다.

어떻게 페이지 테이블의 크기를 줄일 수 있을까?

주요 개념들로는 무엇이 있는가?

새로운 재로 교주들은 어떤 비효율성을 갖는가?

페이징의 두 번째 문제점은 페이지 테이블의 크기이다.

페이지 테이블이 크면 많은 메모리 공간을 차지한다.

배열 형태를 가지는 선형 페이지 테이블(linear page table)을 예로 들어보자.

전술한 바와 같이 선형 페이지 테이블은 상당히 커질 수가 있다.

페이지 크기가 4KB(2^{12} 바이트)이고, 페이지 테이블의 각 항목은 4 바이트인 32 비트 주소 공간(2^{32} 바이트)을 가정해보자.

주소 공간에는 대략 백만 개의 가상 페이지가 존재할 것이다.

여기에 페이지 테이블 항목의 크기를 곱하면(4 바이트 x 백만개), 페이지 테이블의 크기가 된다.

하나의 페이지 테이블 크기가 약 4MB 가 된다.

또 하나 잊으면 안되는 사실이 있다.

일반적으로 각 프로세스는 자기 자신의 페이지 테이블을 갖는다.

백 개의 프로세스가 실행 중이라면 (현대 시스템에서 드물지 않다),

페이지 테이블이 100 개 존재하며, 400MByte 의 메모리가 필요하다.

이런 엄청난 메모리 부담을 해결할 수 있는 기술을 모색해보자.

[1] 간단한 해법: 더 큰 페이지

페이지 테이블의 크기를 간단하게 줄일 수 있는 방법이 한 가지가 있다.

페이지 크기를 증가시키면 된다.

32 비트 주소 공간에서 이번에는 16KB 페이지를 가정해보자.

이제는 18 비트의 VPN 과 14 비트의 오프셋을 갖게된다.

각 PTE(4 바이트)의 크기가 모두 동일하다면, 페이지 테이블에 2^{18} 개의 항목이 있으며, 페이지 테이블의 총 크기는 1MB 가 된다.

기존 페이지 테이블 대비 크기가 1/4 로 감소된다.(당연하다. 테이블의 크기가 1/4 로 감소한 것은 페이지 크기가 정확히 4 배 증가했기 때문이다.)

페이지 크기의 증가는 부작용을 수반한다.

가장 큰 문제는 페이지 내부의 낭비 공간이 증가하는 것이다.

이를 내부 단편화라 한다.(할당된 내부에서 낭비가 발생하기 때문이다.)

응용 프로그램이 여러 페이지를 할당받았지만, 할당받은 페이지의 일부분만 사용하는
터에, 결국 컴퓨터 시스템의 메모리가 금방 고갈되는 현상이 발생한다.

이런 연유에서, 많은 컴퓨터 시스템들이 비교적 작은 페이지들을 사용한다.

일반적으로 4KB(x86) 또는 8KB(SPARCv9)를 사용한다.

페이지 테이블 크기를 감소시키는 우리의 당면 문제는 그리 쉽게 해결되지 않을
것이다.

[1-1] 비교

기본 조건

- | |
|--|
| <ul style="list-style-type: none">- 가상 주소 공간: 32 비트(4GB)- PTE 크기: 4 바이트 |
|--|

(1) 페이지 크기 4KB

- 페이지 크기 = 4KB = 2^{12}
- offset = 12 비트
- VPN = 32 - 12 = 20 비트

페이지 테이블 크기

- 페이지 수 = 2^{20}
- PTE 크기 = 4 바이트
- 총 크기: $2^{20} \times 4 = 2^{22} = 4MB$

(2) 페이지 크기 16KB

- 페이지 크기 = 16KB = 2^{14}
- offset = 14 비트
- VPN = 32 - 14 = 18 비트

페이지 테이블 크기

- 페이지 수 = 2^{18}
- PTE 크기 = 4 바이트
- 총 크기: $2^{18} \times 4 = 2^{20} = 1MB$

* 페이지 크기 = 메모리를 자르는 단위

- 가상 메모리와 물리 메모리를 같은 크기의 블록으로 나누는 기준 크기

* PTE = 페이지 설명서

- 각 페이지가 어디 물리 메모리에 있는지 알려주는 정보

구조

가상 주소 → [VPN | Offset]

VPN → 페이지 테이블 → PTE → PFN

Offset → 그대로 사용

가상 주소 구조(32bit)

[VPN (18bit) | Offset (14bit)]

- VPN → 페이지 번호(페이지 테이블에서 찾음)

- Offset → 페이지 내부 위치(그대로 사용)

가상 메모리 구조(페이지 단위)

- 페이지 크기 = 16KB

가상 메모리

Page 0 (16KB)
Page 1 (16KB)
Page 2 (16KB)
...
Page $2^{18}-1$

- 총 페이지 수 = 2^{18}

페이지 테이블 구조

- 각 페이지마다 하나의 PTE 존재

페이지 테이블

PTE[0]	→ PFN
PTE[1]	→ PFN
PTE[2]	→ PFN
...	
PTE[2 ¹⁸ -1]	→ PFN

- 총 엔트리 = 2^{18}
- 각 엔트리 = 4B

주소 변환 흐름

가상 주소



[VPN | Offset]



페이지 테이블에서 PTE 찾기



PFN 획득



물리 주소 생성

[PFN | Offset]

크기 계산 구조

페이지 테이블 크기 계산

엔트리 개수: 2^{18}

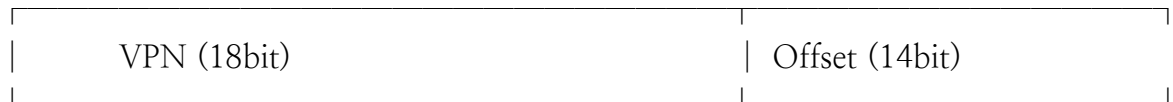
엔트리 크기: 4B

→ 총 크기:

$$2^{18} \times 4 = 2^{20} = 1\text{MB}$$

전체 구조

가상 주소 (32bit)



|



페이지 테이블 (2^{18} entries)

|



PFN 획득

|



물리 주소 = [PFN | Offset]

[2] 하이브리드 접근 방법: 페이징과 세그먼트

인생의 어떤 일을 함에 있어 좋은 방법이 두 가지 있다면, 그 두 방법을 잘 조합하여, 장점만을 취할 수는 없는지도 검토해 보아야 한다.

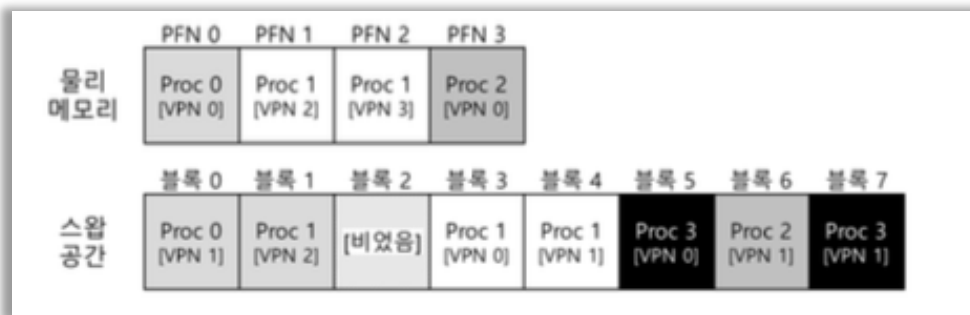
두 가지 방법을 좋아하는 것을 하이브리드(hybrid)라고 부른다.

Multics 의 창시자 Jack Dennis 는 Multics 가상 메모리 시스템을 구축하는 데 있어, 비슷한 아이디어가 떠올랐다.

Dennis 는 페이징과 세그멘테이션을 결합하여 페이지 테이블 크기를 줄이는 아이디어를 내었다.

선형 페이지 테이블의 동작을 면밀히 분석해 보면, 페이징과 세그멘테이션을 효과적으로 결합할 수 있다는 사실을 발견할 수 있다.

힙과 스택에서 실제로 전체 공간 중 작은 부분만 사용되는 경우를 생각해보자.



1KB 페이지들로 이루어진 16KB 주소 공간

PFN	valid	prot	present	dirty
10	1	r-x	1	0
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
23	1	rw-	1	1
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
--	0	---	-	-
28	1	rw-	1	1
4	1	rw-	1	1

16KB 주소 공간을 위한 페이지 테이블

[2-1] 페이지징에서의 주소 공간과 페이지 테이블 낭비 문제

(1) 주소 공간의 의미

주소 공간(Address Space)이란

하나의 프로세스가 사용할 수 있는 전체 가상 메모리 범위를 의미한다.

예를 들어,

“1KB 페이지를 갖는 16KB 주소 공간”이라는 것은 다음을 의미한다.

- 전체 가상 메모리 크기: 16KB
- 페이지 크기: 1KB
- 페이지 수: 16 개(16KB/1KB)

즉, 해당 프로세스는 총 16 개의 가상 페이지를 가질 수 있다.

(2) 페이지 테이블의 구조

페이지징 시스템에서는

각 가상 페이지마다 하나의 페이지 테이블 엔트리(PTE)가 존재한다.

따라서 위 예제에서는:

- 페이지 수: 16 개
- 페이지 테이블 엔트리: 16 개

페이지 테이블은 다음과 같이 구성된다.

페이지 테이블

[0] → 사용 (코드)

[1] → 미사용

[2] → 미사용

[3] → 미사용

[4] → 사용 (힙)

...

[14] → 사용 (스택)

[15] → 사용 (스택)

(3) 문제: 페이지 테이블 낭비

실제 프로그램은 주소 공간 전체를 사용하지 않는다.

예제에서 실제로 사용되는 페이지는 다음과 같다.

- 코드: VPN 0
- 힙: VPN 4
- 스택: VPN 14, 15

총 4 개 페이지만 사용

하지만 페이지 테이블은 전체 16 개 페이지에 대해 모두 엔트리를 가져야 한다.
즉, 사용하지 않는 페이지에도 PTE 가 존재 → 메모리 낭비 발생

이러한 낭비는 작은 주소 공간에서도 발생하며,
32 비트(4GB)와 같은 큰 주소 공간에서는 훨씬 심각해진다.

(4) 문제의 본질

페이지 테이블은 다음과 같은 특징을 가진다.

- 가상 주소 공간 전체를 기준으로 생성됨
- 실제 사용 여부와 무관하게 모든 페이지를 포함

즉, 이론적인 주소 공간 전체를 관리하기 때문에 낭비가 발생한다.

(5) 해결 아이디어: 세그멘테이션 + 페이징(하이브리드)

이 문제를 해결하기 위해 제안된 방법이 세그멘테이션과 페이징을 결합한 방식이다.

기본 아이디어

전체 주소 공간을 하나로 보지 않고, 다음과 같이 논리적으로 나눈다.

- 코드/힙/스택

그리고 각 세그먼트마다 별도의 페이지 테이블을 둔다.

하드웨어 구조

각 세그먼트마다 다음 정보를 유지한다.

- Base: 해당 세그먼트 페이지 테이블의 시작 주소
- Bound: 해당 페이지 테이블의 크기(유효 범위)

동작 방식

가상 주소는 다음과 같이 구성된다.

[세그먼트 번호(SN) VPN Offset]

주소 변환 과정:

1. 세그먼트 번호(SN)로 어떤 세그먼트인지 결정
2. 해당 세그먼트의 Base 레지스터 선택
3. VPN 을 이용해 페이지 테이블에서 PTE 찾기
4. PFN + Offset → 물리 주소 생성

(6) 장점

- 사용되는 세그먼트만 페이지 테이블 생성
- 불필요한 페이지 테이블 공간 제거
- 메모리 사용 효율 증가

특히, 드문드문 사용되는 주소 공간에서 효과적

(7) 한계

하지만 이 방식도 완벽하지 않다.

문제점

1. 세그멘테이션 의존성
 - 프로그램이 코드/힙/스택 구조를 따라야 함
 - 유연성 감소
2. 외부 단편화 발생
 - 세그먼트 크기가 가변적
 - 메모리 관리 복잡
3. 여전히 낭비 가능
 - 힙처럼 sparse 한 경우 여전히 비효율

(8) 최종 정리

주소 공간은 프로세스가 사용할 수 있는 전체 가상 메모리 범위이며, 페이지 테이블은 이 전체 공간을 기준으로 생성하기 때문에 실제로 사용하지 않는 영역에서도 메모리 낭비가 발생한다.

이를 해결하기 위해 세그멘테이션과 페이징을 결합한 하이브리드 방식이 등장했지만, 이 또한 외부 단편화와 구조적 제약이라는 새로운 문제를 가진다.

[3] 멀티 레벨 페이지 테이블

세그멘테이션을 사용하지 않고 페이지 테이블 크기를 줄이는 방법에 대해서 생각해보자.

어떻게 하면 사용하지 않는 주소 공간을 페이지 테이블에서 제거할 수 있을까?

이번에 소개할 기법은 멀티 레벨 페이지 테이블이다.

멀티 레벨 페이지 테이블에서는 선형 페이지 테이블을 트리 구조로 표현한다.

매우 효율적이기 때문에 많은 현대 시스템에서 사용되고 있다.

이 기법을 좀 더 상세히 설명하도록 하겠다.

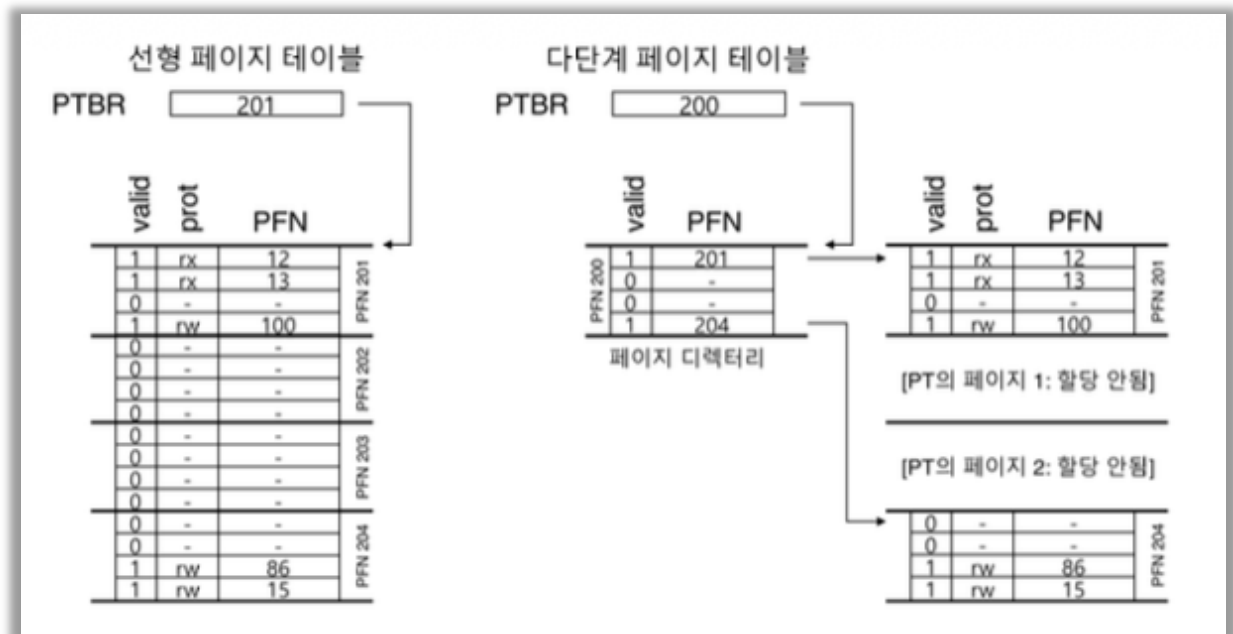
멀티 레벨 페이지 테이블의 기본 개념은 간단하다.

먼저, 페이지 테이블을 페이지 크기의 단위로 나눈다.

그 다음, 페이지 테이블의 페이지가 유효하지 않은 항목만 있으면, 해당 페이지를 할당하지 않는다.

페이지 디렉터리(page directory)라는 자료 구조를 사용하여 페이지 테이블 각 페이지의 할당 여부와 위치를 파악한다.

페이지 디렉터리는 페이지 테이블을 구성하는 각 페이지의 존재 여부와 위치 정보를 가지고 있다.(페이지 디렉터리는 하나의 프로세스 당 하나)



그림의 예제를 보자.

좌측 그림은 전형적인 선형 페이지 테이블이다.

페이지 테이블의 중앙부에 해당하는 주소 공간은 사용되고 있지 않다.

그러나 페이지 테이블에서 항목들이 할당되어 있다(페이지 테이블의 가운데 두 페이지).

우측은 동일한 주소 공간을 다루는 멀티 레벨 페이지 테이블이다.

페이지 디렉터리에는 두 개의 유효한 페이지가 있다.(첫번째와 마지막)

유효 페이지 두 개는 메모리에 존재한다.

이 예를 통해 멀티 레벨 페이지 테이블의 동작을 좀 더 쉽게 알 수 있다.

선형 페이지 테이블에서 사용되었던 페이지들이 더 이상 필요없고, 페이지 디렉터리를 이용하여 페이지 테이블의 어떤 페이지들이 할당되었는지를 관리한다.

간단한 2 단계 테이블에서, 페이지 디렉터리의 각 항목은 페이지 테이블의 한 페이지를 나타낸다.

페이지 디렉터리는 페이지 디렉터리 항목(page directory entries, PDE)들로 구성된다.

각 항목(PDE)의 구성은 페이지 테이블의 각 항목(Page Table Entry)과 유사하다.

유효(valid)비트와 페이지 프레임 번호(page frame number, PFN)를 갖고 있다.

실제 구현에 따라 추가 구성 요소가 존재할 수 있다.

하지만, PTE의 유효 비트와 PDE의 유효 비트는 약간 다르다.

PDE 항목이 유효하다는 것은, 그 항목이 가리키고 있는 (PFN을 통해서) 페이지들 중 최소한 하나가 유효하다는 것을 의미한다.

즉, PDE가 가리키고 있는 페이지 내의 최소한 하나의 PTE의 valid bit가 1로 설정되어 있다.

만약 PDE의 항목이 유효하지 않다면(즉, 0이라면), PDE는 실제 페이지가 할당되어 있지 않은 것이다.

멀티 레벨 페이지 테이블은 이제까지 언급된 다른 기법들에 비해 몇 가지 장점이 있다.

첫째, 멀티 레벨 테이블은 사용된 주소 공간의 크기에 비례하여 페이지 테이블 공간이 할당된다.

그렇기 때문에, 보다 작은 크기의 페이지 테이블로 주소 공간을 표현할 수 있다.

두 번째, 페이지 테이블을 페이지 크기로 분할함으로써 메모리 관리가 매우 용이하다.

페이지 테이블을 할당하거나 확장할 때, 운영체제는 free 페이지 풀에 있는 빈 페이지를 가져다 쓰면된다.

멀티 레벨 페이징을 단순한 선형 페이지 테이블 방식과 비교해 보자.

선형 페이지 테이블의 각 항목은 해당 가상 페이지의 물리 페이지 주소를 가지고 있다.(즉, 디스크로 스왑되지 않는다)

선형 페이지 테이블은 연속된 물리 메모리 공간을 차지한다.

큰 페이지 테이블(4MB 라고 하자)의 경우, 해당 크기의 연속된 빈 물리 페이지를 찾는 것이 쉽지 않다.

멀티 레벨 페이징에서는 페이지 디렉터리를 사용하여 각 페이지 테이블 페이지들의 위치를 파악한다.

페이지 테이블의 각 페이지들이 물리 메모리에 사재해 있더라도 페이지 디렉터리를 이용하여 그 위치를 파악할 수 있으므로, 페이지 테이블을 위한 공간 할당이 매우 유연하다.

한 가지 유의해야 할 사항이 있다.

멀티 레벨 테이블에는 추가 비용이 발생한다.

TLB 미스 시, 주소 변환을 위해 두 번의 메모리 로드가 발생한다(페이지 디렉터리와 PTE 접근을 위해 각각 한 번씩).

선형 페이지 테이블에서는 한 번의 접근만으로 주소 정보를 TLB 로 탑재한다.

멀티 레벨 테이블은 시간(페이지 테이블 접근 시간)과 공간(페이지 테이블 공간)을 상호 절충(time-space-trade-offs)한 예라 할 수 있다.

페이지 테이블 크기를 줄이는 데 성공하였으나, 대신 메모리 접근 시간이 증가했다. TLB 히트시(대부분 메모리 접근은 TLB 히트이다) 성능은 동일하지만, TLB 미스 시에는 두 배의 시간이 소요된다.

또 하나의 단점은 복잡도이다.

페이지 테이블 검색이 단순 선형 페이지 테이블의 경우보다 더 복잡해진다.

검색을 하드웨어로 구현하느냐 혹은 운영체제로 구현하느냐 여부와는 무관하다.

대부분의 경우, 성능 개선이나 부하 경감을 위해, 우리는 보다 복잡한 기법을 도입한다.

멀티 레벨 페이지 테이블의 경우에는 메모리 자원의 절약을 위해, 페이지 테이블 검색을 좀 더 복잡하게 만들었다.

[3-1] 멀티 레벨 페이징 예제

멀티 레벨 페이지 테이블의 개념을 이해하기 위해서 예제를 하나 살펴보자.

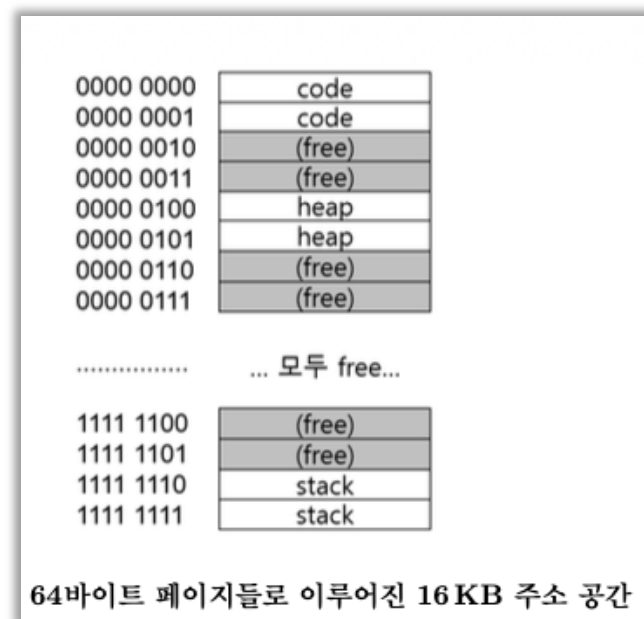
64 바이트 페이지를 갖는 16KB 크기의 작은 주소 공간(가상 메모리)을 생각 해보자.

14 비트 가상 주소 공간이다.

VPN 8 비트, 페이지 오프셋에 6 비트가 필요하다.

선형 페이지 테이블은 $2^8(256)$ 개 엔트리로 구성된다.

주소 공간에서 작은 부분만 사용된다 하더라도, 선형 페이지 테이블의 크기는 변하지 않는다.



그림은 이 구조를 갖는 주소 공간의 예시를 나타낸다.

이 예제에서는, 가상 페이지 0 과 1 은 코드, 가상 페이지 4 와 5 는 힙 그리고 가상 페이지 256 와 255 는 스택으로 사용된다.

주소 공간의 나머지 페이지들은 미사용 중이다.

이 주소 공간을 2 단계 페이지 테이블로 구성해 보자.

선형 페이지 테이블을 페이지 단위로 분할한다.

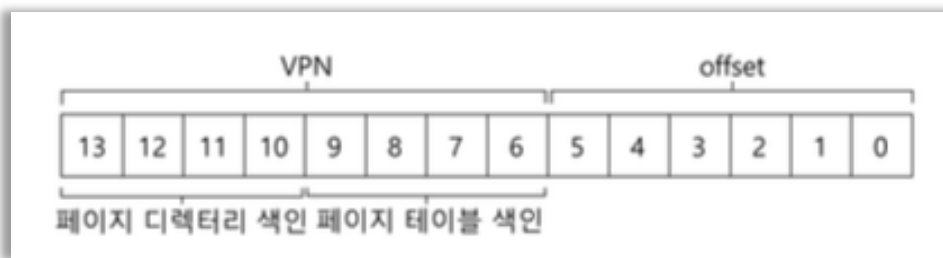
전체 테이블은(이 예제에서) 총 256 개의 항목을 갖고 있다는 것을 상기하자.

각 PTE 는 4 바이트라 가정한다.

페이지 테이블의 크기는 1KB(256×4 바이트)이다.

페이지가 64 바이트라고 하면 1KB 의 페이지 테이블은 16 개의 64 바이트 페이지들로 분할된다.

각 페이지에는 16 개의 PTE 가 있다.



이제
VPN 으로부터
페이지
디렉터리
인덱스를
추출하고,

페이지 테이블의 각 페이지 위치를 파악하는 법을 살펴보자.

페이지 디렉터리, 페이지 테이블의 페이지들 모두 항목의 배열이라는 것을 기억해야 한다.

VPN 을 이용하여 인덱스를 구성하는 법만 찾으면 된다.

먼저 페이지 디렉터리의 인덱스를 만들어보자.

예제의 작은 페이지 테이블은 256 개의 항목으로 16 개의 페이지로 나뉘어 있다.

페이지 디렉터리는 페이지 테이블의 각 페이지마다 하나씩 있어야 하기 때문에 총 16 개의 항목이 있어야 한다.

결과적으로 VPN 의 4 개의 비트를 사용하여 디렉터리를 구성하며, 여기서는 VPN 의 상위 4 비트를 다음과 같이 사용한다.

VPN 에서 페이지-디렉터리 인덱스(page-directory index, 짧게 PDIndex 라고 하자)를 추출하고 나면

$PDEAddr = PageDirBase + (PDIndex * sizeof(PDE))$ 라는 간단한 식을 사용하여 페이지-디렉터리 항목(page-directory entry, PDE)의 주소를 찾을 수 있다.

이렇게 페이지 디렉터리가 구성이 되며, 이것을 활용하여 계속해서 주소 변환 과정을 분석하도록 한다.

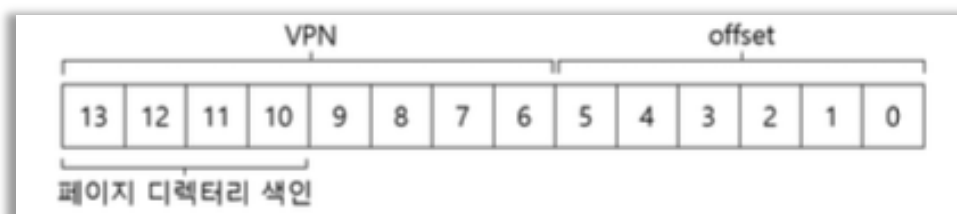
페이지-디렉터리의 해당 항목이 무효(invalid)라고 표시되어 있으면, 이 주소 접근은 유효하지 않다. 예외가 발생한다.

해당 PDE 가 유효하다면 추가 작업을 해야 한다.

구체적으로 살펴보자.

이 페이지 디렉터리 항목이 가리키고 있는 페이지 테이블의 페이지에서 원하는 페이지 테이블 항목(page table entry, PTE)을 읽어 들이는 것이 목표다.

이 PTE 를 찾기 위해서 VPN 의 나머지 비트들을 사용한다.



PTE 의 주소를 다음과 같이 계산한다.

$$\text{PTEAddr} = (\text{PDE.PFN} \ll \text{SHIFT}) + (\text{PTIndex} * \text{sizeof(PTE)})$$

PTE의 주소를 생성하기 위해서 페이지-디렉터리 항목에서 얻은 페이지-프레임 번호(page-frame number, PFN)를 먼저 좌측 쉬프트 연산하고 그 값을 페이지 테이블 인덱스에 합산한다.

이 모든 것이 제대로 동작하는지를 보기 위해, 멀티 레벨 페이지 테이블에 실제 값들을 넣은 후에 하나의 가상 주소를 변환해 보자.

페이지 디렉터리		PT의 페이지 (@PFN:100)			PT의 페이지 (@PFN:101)		
PFN	유효한가?	PFN	유효	prot	PFN	유효	prot
100	1	10	1	r-x	---	-	---
---	0	23	1	r-x	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	80	1	rw-	---	0	---
---	0	59	1	rw-	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	--	0	---	---	0	---
---	0	---	0	---	55	1	rw-
101	1	---	0	---	45	1	rw-

이 예제를 위해 페이지 디렉터리에서부터 시작해보자.

이 그림에서 각 페이지 디렉터리 항목(PDE)은 주소 공간의 페이지 테이블의 페이지에 대해 무엇인가를 기술하고 있다.

이 예제에서는 주소 공간에 두 개의 유효한 영역이 있으며(처음과 끝), 그리고 그 사이는 무효한 맵핑들이 존재한다.

물리 페이지 100 에 (페이지 테이블의 0 번째 페이지의 물리 프레임 번호), 페이지 테이블의 첫 16 개 항목이 존재한다.

이 항목들은 가상 주소 공간의 첫 16 개 페이지에 대한 물리 주소를 가진다.

그림의 가운데 열이 해당 페이지의 내용이다.

페이지 테이블의 첫 페이지에는 첫 16 개의 VPN 과 물리 페이지 주소가 있다.

이 예제에서는 VPN 0 과 1 이 유효하고(코드 세그먼트), 4 와 5 도 유효하다(힙), 그러므로 테이블에는 각 페이지들의 매핑 정보가 있다.

그 외의 나머지 엔트리들은 무효로 표기되어 있다.

PFN 101 에 다른 유효 페이지들에 대한 정보가 들어 있다.

이 페이지에는 가상 주소 공간의 마지막 16 개의 VPN 에 대한 매핑이 담겨 있다. 그림의 우측에 상세 내용이 있다.

이 예제에서는, VPN254 와 255 가 유효 페이지이다.

이들은 스택에 해당한다.

이 예제를 통해서 멀티 레벨 인덱스 구조가 공간을 얼마나 절약할 수 있는지를 확인할 수 있다.

선형 페이지 테이블의 열여섯 개의 페이지들을 모두 할당하는 대신 단지 세 개만 할당하였다.

페이지 디렉터리를 위해서 한 페이지, 그리고 유효한 매핑 정보를 갖고 있는 페이지 테이블 내의 두 부분을 위해 두 페이지를 할당하였다.

더 큰 (32 비트 또는 64 비트) 주소 공간에서는 더 많은 공간이 절약된다.

마지막으로 최종 주소 변환을 해보자.

VPN 254 의 0 번째 바이트를 가리키는 주소는 다음과 같다.

0x3F80 또는 이진수로 11 1111 1000 0000 이다.

페이지 디렉터리 내에서 각 항목을 가리키기 위해 VPN 의 상위 4 비트를 사용하였다.

1111 은 페이지 디렉터리의 마지막 엔트리다(0 부터 시작했다면 15 번째가 된다).

페이지 디렉터리의 15 번째 엔트리에는 물리 프레임 주소 101 이 저장되어 있다.

VPN 의 다음의 4 비트를 (1110)을 인덱스로 사용하여, 페이지 테이블의 해당 페이지에서 원하는 PTE 를 찾는다.

1110 는 101 번 페이지의 16 개 항목 중에 마지막에서 두 번째 항목이다.

여기에 55 가 저장되어 있다.

종합하면 가상주소 페이지 254(1111 1110)는 물리 페이지 55 에 존재한다.
오프셋 0000000 과 PFN 55(또는 헥사로 0x37)를 결합하여 다음과 같이 물리 주소를 구할 수 있으며, 해당 메모리를 접근하는 데 사용된다.

$\text{PhysAddr} = (\text{PET.PFN} \ll \text{SHIFT}) + \text{offset} = 00\ 1101\ 1100\ 0000 = 0x0DC0$

이제 페이지 디렉터리를 사용하여 페이지 테이블의 페이지를 가리키는 2 단계 페이지 테이블에 대한 개념이 어느정도 이해되었을 것이다.

곧 구체화하겠다.

2 단계 페이지 테이블이 충분하지 않을 때가 있다!

[3-2] 2 단계 이상 사용하기

지금까지는 멀티 레벨 페이지 테이블은 페이지 디렉터리와 페이지 테이블의 2 개 단계를 가정하였다.

경우에 따라서 트리의 단계를 더 증가시키는 것도 가능하다(그리고 사실, 그래야 할 필요가 있다).

간단한 예제를 통해 2 단계 이상의 멀티 레벨 테이블에 대해 알아보자.

이 예제에서는 512 바이트 페이지와 30 비트 가상 주소 공간을 가정한다.

가상 주소는 21 비트의 가상 페이지 번호와 9 비트의 오프셋을 갖게 된다.

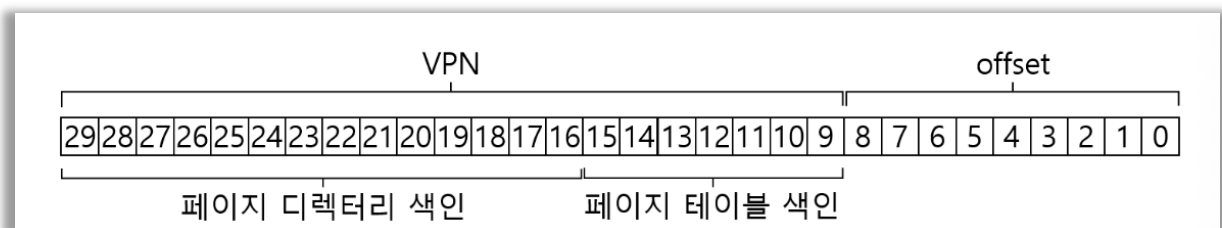
멀티 레벨 페이지 테이블의 목적은 페이지 테이블의 모든 분할된 부분들이 단일 페이지 크기에 맞도록 하는 것이다.

만약 페이지 디렉터리가 너무 커지면, 어떻게 될까?

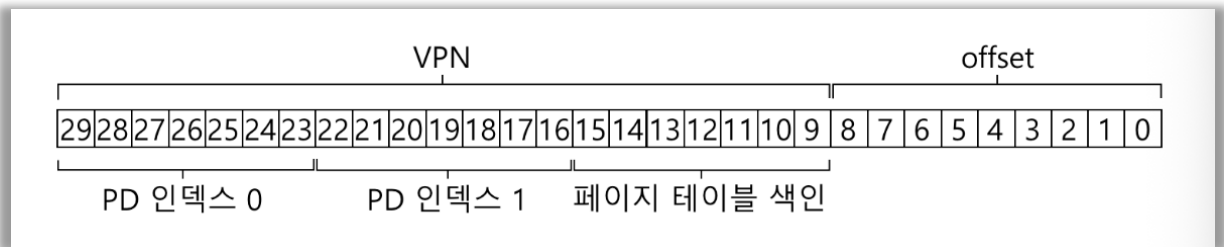
멀티 레벨 테이블에서 몇 단계를 둘지 정하기 위해서는 먼저 한 페이지에 몇 개의 페이지 테이블 항목을 저장할 수 있을지를 계산해야 한다.

페이지 크기가 512 바이트이고 PTE 의 크기가 4 바이트라고 가정하면 한 페이지에 128 개의 PTE 를 넣을 수 있다.

페이지 테이블의 페이지를 인덱스로 쓰려면, VPN 의 하위 7 비트($\log_2 128$)가 필요하다.



페이지 디렉터리를 위해서 몇 개의 비트가 남았는지를 이 그림에서 알 수 있다.
 14 개의 비트가 남는다.
 2 단계 페이지를 사용한다면, 페이지 디렉터리에 2^{14} 개의 항목이 있게 된다.
 페이지 디렉터리를 위해서 128 페이지 분량의 연속된 메모리가 필요하다.
 페이지 테이블을 페이지 단위로 나누어 배치할 수 있도록 하는 멀티 레벨 페이지 테이블의 근본 취지가 훼손된 셈이다.



이 문제를 해결하기 위해서, 페이지 디렉터리 자체를 멀티 페이지들로 나누어서 트리의 단계를 늘리도록 한다.
 그리고 페이지 디렉터리의 페이지들을 가리킬 수 있도록 그 위에 새로운 페이지 디렉터리를 추가한다.

결과적으로 다음과 같이 가상 주소를 분할할 수 있다.
 이제 가상 주소의 최상위 비트들을 (그림에서 PD Index 0)을 사용하여 상위 단계의 페이지 디렉터리에서 엔트리를 찾는다.
 이 인덱스를 사용하여 상위 단계 페이지 디렉터리에서 페이지 디렉터리 항목을 가져온다.
 만약 유효하다면, 상위 단계 페이지 디렉터리에서 얻은 물리 주소와 두 번째 단계의 페이지 디렉터리 인덱스(PD Index 1)를 결합하여 페이지 테이블 인덱스가 존재한 물리 페이지를 구한다.

해당 페이지가 유효할 경우, 최종적으로 PTE 주소는 2 번째 단계의 페이지 디렉터리 항목에서 얻은 페이지 테이블의 물리 주소와 페이지 테이블 인덱스를 결합하여 구한다.

실제 주소를 구하는데 드는 작업이 상당하다.
 멀티 레벨 페이지 테이블 사용 시 수반되는 작업이기도 하다.


```

1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)          // TLB 히트
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                          // TLB 미스
11     // 먼저, 페이지 디렉터리 항목을 얻어야 함
12     PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13     PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14     PDE = AccessMemory(PDEAddr)
15     if (PDE.Valid == False)
16         RaiseException(SEGMENTATION_FAULT)
17     else
18         // PDE 가 유효함: 이제 페이지 테이블에서 PTE를 가져오자
19         PTIndex = (VPN & PT_MASK) >> PT_SHIFT
20         PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
21         PTE = AccessMemory(PTEAddr)
22         if (PTE.Valid == False)
23             RaiseException(SEGMENTATION_FAULT)
24         else if (CanAccess(PTE.ProtectBits) == False)
25             RaiseException(PROTECTION_FAULT)
26         else
27             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
28             RetryInstruction()

```

〈그림 23.6〉 멀티 레벨 페이지 테이블의 제어 흐름

[3-3] 변환 과정: TLB 를 기억하자

2 단계 페이지 테이블 사용 시, 전체 주소 변환 과정을 알고리즘 형태로 요약 정리해보자.

이 그림은 모든 페이지 참조에 대해 하드웨어가 어떤 식으로 동작하는지를 나타낸다(하드웨어 기반 TLB 를 가정).

그림에서 볼 수 있듯이, 복잡한 멀티 레벨 페이지 테이블 접근을 거치기 전에, 우선 TLB 를 검사한다.

히트가 되면 페이지 테이블을 참조 없이 물리 주소를 직접 구성한다.

TLB 미스 시에만, 멀티 레벨 페이지 테이블의 모든 단계를 거쳐 물리 주소를 구하게 된다.

이 알고리즘을 통해 TLB 미스 발생 시, 전통적인 2 단계 페이지 테이블의 주소 계산 비용을 볼 수 있다.

주소 변환을 위해 두 번의 추가 메모리 접근이 발생한다.

[4] 역 페이지 테이블

좀 더 획기적인 공간 절약 방법으로 역 페이지 테이블(inverted page table)이 있다. 이 방법에서는 여러 개의 페이지 테이블(시스템의 프로세스당 하나씩) 대신 시스템에 단 하나의 페이지 테이블만 둔다.

페이지 테이블은 물리 페이지를 가상 주소 상의 페이지로, 변환한다.

역 페이지 테이블의 각 항목은 해당 물리 페이지를 사용 중인 프로세스 번호, 해당 가상 페이지 번호를 갖고 있다.

페이지 테이블의 목적은 가상 주소를 물리 주소로 변환하는 것이다.

역 페이지 테이블에서는 주소 변환을 위해 전체 테이블을 검색해서 원하는 가상 주소 페이지를 갖는 항목을 찾아야 한다.

순차 탐색은 느리다.

탐색 속도 향상을 위해 주로 해시 테이블을 사용한다.

PowerPC 는 이 구조를 사용하는 예이다.

좀 더 일반적인 시각에서 보자면, 역 페이지 테이블 역시 하나의 자료 구조일 뿐이다. 자료 구조로 다양한 시도를 할 수 있다.

예를 들면, 작게도 만들고 크게도 만들 수 있으며 느리게도 빠르게도 만들 수가 있다. 멀티 레벨과 반전된 페이지 테이블들은 할 수 있는 다양한 방법 중 두 가지 예일 뿐이다.

[5] 페이지 테이블을 디스크로 스와핑하기

마지막으로 중요한 가정을 해제하겠다.

이제까지는 페이지 테이블이 커널이 소유하고 있는 물리 메모리 영역에 존재한다고 가정하였다.

페이지 테이블 크기 축소를 위해 많은 시도를 하더라도, 여전히 모든 페이지 테이블을 메모리에 상주시키기에는 양이 너무 클 수도 있다.

그렇기 때문에 어떤 시스템들은 페이지 테이블들을 커널 가상 메모리에 존재시키고, 시스템의 메모리가 부족할 경우, 페이지 테이블들을 디스크로 스왑(swap)하기도 한다.

이 부분에 대해서는 페이지들을 메모리에 탑재하고 제거하는 방법을 이해한 후에, 다시(구체적으로 VAX/VMS 에 대한 사례 연구를 다루는 장에서) 상세히 다루도록 한다.

[1-4] 가상화: 물리 메모리 크기의 극복_메커니즘

핵심 질문: 물리 메모리 이상으로 나아가기 위해서 어떻게 할까
운영체제는 어떻게 크고 느린 장치를 사용하면서 마치 커다란 가상 주소 공간이 있는 것처럼 할 수 있을까?

지금까지 가상 주소 공간이 비현실적으로 작아서 모두 물리 메모리에 탑재가 가능한 것으로 가정하였다.

사실 실행 중인 프로세스의 전체 주소 공간이 메모리에 탑재된 것으로 가정하고 있었다. 이제 그 가정을 완화한다.

우리는 다수 프로세스들이 동시에 각자 큰 주소 공간을 사용하고 있는 상황을 가정한다.

이를 위해, 메모리 계층에 레이어의 추가가 필요하다.

지금까지는 모든 페이지들이 물리 메모리에 존재하는 것을 가정하였다.

하지만 큰 주소 공간을 지원하기 위해서 운영체제는 주소 공간 중에 현재는 크게 필요하지 않은 일부를 보관해 둘 공간이 필요하다.

일반적으로 그 공간은 메모리 공간보다 더 크며, 더 느리다(만약 더 빠르다면 그것을 메모리로 사용할 것이다.).

현대 시스템에서는 보통 하드디스크 드라이브가 이 역할을 담당한다.

이제 메모리 계층에서 크고 느린 하드디스크 드라이브가 가장 하부에 위치하고, 그 위에 메모리가 있다.

한 가지 짚고 넘어가야 할 것은, 왜 프로세스에게 굳이 “큰”주소 공간을 제공해야 하는가이다.

이에 대한 답은 다시 한 번 편리함과 사용 용이성이다.

주소 공간이 충분히 크면, 프로그램의 자료 구조들을 위한 충분한 메모리 공간이 있는지 걱정하지 않아도 된다.

필요 시 메모리 할당을 운영체제에게 요청하기만 하면 된다.

운영체제가 이러한 “가상” 환경을 제공하면, 인생이 편해진다.

이와 대비되는 기법으로 과거 시스템에서 사용되던 메모리 오버레이(memory overlay)가 있다.

이 시스템에서는 프로그래머가 코드 또는 데이터의 일부를 수동으로 메모리에 탑재/제거했다.

상상만해도 힘들다.

함수 호출이나 데이터 접근 시, 코드 또는 데이터를 먼저 메모리에 탑재해야 한다.

스왑 공간이 추가되면 운영체제는 실행되는 각 프로세스들에게 큰 가상 메모리가 있는 것 같은 환상을 줄 수 있다.

멀티프로그래밍 시스템(컴퓨터의 사용률을 높이기 위해 “동시에” 여러 프로그램들을 실행시키는 시스템)이 발명되면서 많은 프로세스들의 페이지를 물리 메모리에 전부 저장하는 것이 불가능하게 되었다.

그래서 일부 페이지들을 스왑 아웃하는 기능이 필요하게 되었다.

멀티프로그래밍과 사용 편의성 등의 이유로 실제 물리 메모리보다 더 많은 용량의 메모리가 필요하게 되었다.

이것이 현대 Virtual Memory(가상 메모리)의 역할이다.

이제 그 방식을 배워보도록 하겠다.

[1] 스왑 공간

가장 먼저할 일은 디스크에 페이지들을 저장할 수 있는 일정 공간을 확보하는 것이다.

이 용도의 공간을 스왑 공간(swap space)이라고 한다.

스왑 공간이라 불리는 이유는 메모리 페이지를 읽어서 이곳에 쓰고(swap out), 여기서 페이지를 읽어 메모리에 탑재 시키기(swap in) 때문이다.

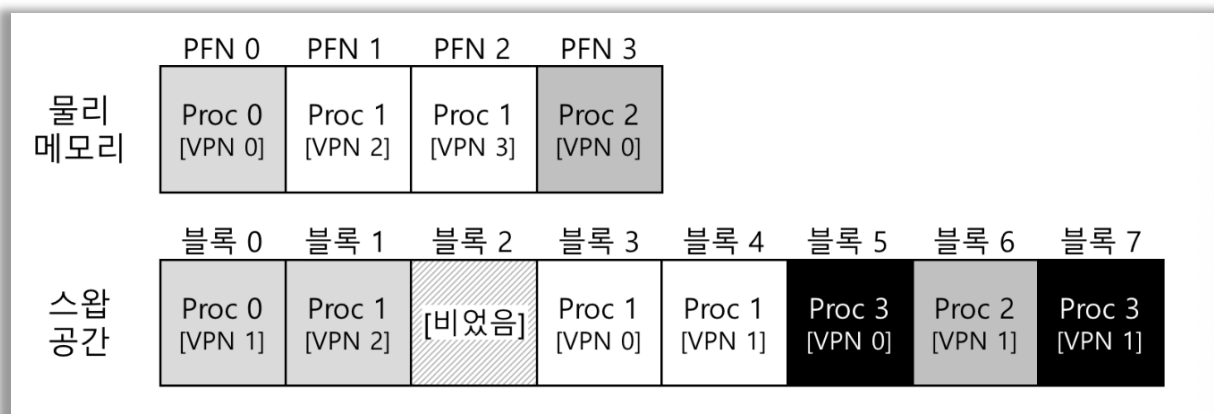
스왑 공간의 입출력 단위는 페이지라고 가정한다.

운영체제는 스왑 공간에 있는 모든 페이지들의 디스크 주소를 기억해야 한다.

스왑 공간의 크기는 매우 중요하다.

시스템이 사용할 수 있는 메모리 페이지의 최대 수를 결정하기 때문이다 .

일단 문제를 단순화하기 위해, 스왑 공간은 매우 크다고 가정하자.



간단한 예제를 보도록 하자.

물리 메모리와 스왑 공간에는 각각 4 개의 페이지와 8 개의 페이지를 위한 공간이 존재한다.

이 예에서는 세 개의 프로세스(Proc 0, Proc1, 그리고 Proc2)가 물리 메모리를 공유하고 있다.

이들 세 프로세스는 몇 개의 유효한 페이지들만 메모리에 올려 놓았으며 나머지 페이지들은 디스크에 스왑 아웃되어 있다.

네 번째 프로세스(Proc3)의 모든 페이지들은 디스크로 스왑 아웃되어 있기 때문에 현재 실행 중이 아닌 것이 분명하다.

스왑 영역에 하나의 블록이 비어 있다.

예제를 통해, 스왑 공간을 이용하면, 시스템에 실제 물리적으로 존재하는 메모리 공간보다 더 많은 공간이 존재하는 것처럼 가장할 수 있다는 것을 알 수 있다.

스왑 공간에만 스왑을 할수 있는 것은 아니라는 사실에 유념하자.

예를 들어, 프로그램을 실행한다고 하자.(예) ls 또는 당신이 컴파일한 main 프로그램) 이 실행 프로그램의 페이지들은 디스크에 존재한다.

프로그램이 실행되면 각 페이지들은 메모리로 탑재된다.(프로그램 실행 시 한 번에 모두 탑재되거나 또 필요 시 한 페이지씩 탑재할 수 있다.)

물리 메모리에 추가 공간을 확보해야 할 때, 코드 영역의 페이지들이 차지하는 물리 페이지는 즉시 다른 페이지가 사용할 수 있다.

코드가 저장되어 있는 파일 시스템 영역이 스왑 목적으로 사용되는 셈이다.

해당 페이지들은 디스크에 원본이 있으므로 언제든지 다시 스왑-인이 가능하기 때문이다.

[2] Present Bit

디스크에 스왑 공간을 확보했으니 이제 페이지 스왑을 위한 기능을 다룰 차례다. 하드웨어 기반의 TLB 를 사용하는 시스템을 가정하자.

먼저 메모리가 참조되는 과정을 상기해 보자.

프로세스가 가상 메모리 참조를 생성한다(명령어 탑재나 데이터 접근 등).

하드웨어는 메모리에서 원하는 데이터를 가져오기 전에, 우선 가상 주소를 물리 주소로 변환한다.

하드웨어는 먼저 가상 주소에서 VPN 을 추출한 후에 TLB 에 해당 정보가 있는지 검사한다.(TLB 히트)

만약 히트가 되면 물리 주소를 얻은 후에 메모리로 가져온다. 매우빠르다.

대부분의 경우가 여기에 해당하기를 바란다(추가적인 메모리 접근이 필요 없다.)

만약 VPN 을 TLB 에서 찾을 수 없다면(즉, TLB 미스), 하드웨어는 페이지 테이블의 메모리 주소를 파악하고 (페이지 테이블 베이스 레지스터를 사용), VPN 을 인덱스로 하여 원하는 페이지 테이블 항목(PTE)을 추출한다.

해당 페이지 테이블 항목이 유효하고, 관련 페이지가 물리 메모리에 존재하면 하드웨어는 PTE 에서 PFN 정보를 추출하고, 그 정보를 TLB 에 탑재한다. TLB 탑재 후 명령어를 재실행한다. 이번에는 TLB 에서 히트된다. 여기까지는 순조롭다.

페이지 디스크로 스왑되는 것을 가능케 하려면, 많은 기법들이 추가되어야 한다. 특히, 하드웨어가 PTE 에서 해당 페이지가 물리 메모리에 존재하지 않는다는 것을 표현해야 한다.

하드웨어는(또는 소프트웨어가 관리하는 TLB 기법인 경우에는 운영체제가) present bit 를 사용하여 각 페이지 테이블 항목에 어떤 페이지가 존재하는지를 표현한다.

Present 비트가 1 로 설정되어 있다면, 물리 메모리에 해당 페이지가 존재한다는 것이고 위에 설명한 대로 동작한다.

만약 그 비트가 0 으로 설정되어 있다면, 메모리에 해당 페이지가 존재하지 않고 디스크 어딘가에 존재한다는 것을 나타낸다.

물리 메모리에 존재하지 않는 페이지를 접근하는 행위를 일반적으로 페이지 폴트(page fault)라 한다.

페이지 폴트가 발생하면, 페이지 폴트를 처리하기 위해 운영체제로 제어권이 넘어간다.

페이지 폴트 핸들러(page-fault handler)가 실행된다.

그 세부내용을 다음에서 살펴보도록 하자.

[3] 페이지 폴트

TLB 미스의 처리 방법에 따라, 두 종류의 시스템이 있었다.

하드웨어 기반의 TLB(하드웨어가 페이지 테이블을 검색하여 원하는 변환 정보를 찾는 것)와 소프트웨어 기반의 TLB(운영체제가 처리하는 것)이다.

둘 중 어느 것이든, 페이지 폴트가 발생하면, 운영체제가 그 처리를 담당한다.

운영체제의 페이지 폴트 핸들러가 그 처리 메커니즘을 규정한다.

거의 대부분의 시스템들에서 페이지 폴트는 소프트웨어적으로 처리된다.

하드웨어 기반의 TLB도 페이지 폴트 처리는 운영체제가 담당한다.

만약 요청된 페이지가 메모리에 없고, 디스크로 스왑되었다면, 운영체제는 해당 페이지를 메모리로 스왑해 온다.

자연적으로 등장하는 질문이 있다.

원하는 페이지의 위치를 어떻게 파악할지?

많은 시스템들에서 해당 정보 - 즉, 해당 페이지의 스왑 공간상에서의 위치-를 페이지 테이블에 저장한다.

운영체제는 PFN과 같은 PTE 비트들을 페이지의 디스크 주소를 나타내는데 사용할 수 있다.

페이지 폴트 발생 시, 운영체제는 페이지 테이블 항목에서 해당 페이지의 디스크 상 위치를 파악하여, 메모리로 탑재한다.

디스크 I/O가 완료되면 운영체제는 해당 페이지 테이블 항목(PTE)의 PFN 값을 탑재된 페이지의 메모리 위치로 갱신한다.

이 작업이 완료되면 페이지 폴트를 발생시킨 명령어가 재실행된다.

재실행으로 인해 TLB 미스가 발생될 수 있다.

TLB 미스 처리 과정에서 TLB 값이 갱신된다.(이를 피하기 위한 대안으로 페이지 폴트를 처리 시 함께 TLB를 갱신하도록 할 수도 있다.)

최종적으로, 마지막 재실행 시에 TLB에서 주소 변환 정보를 찾게 되고, 이를 이용하여 물리 주소에서 원하는 데이터나 명령어를 가져온다.

I/O 전송 중에는 해당 프로세스가 차단된(blocked) 상태가 된다는 것을 유의해야 한다.

페이지 폴트 처리 시 운영체제는 다른 프로세스들을 실행할 수 있다.

I/O 실행은 매우 시간이 많이 소요되기 때문에 한 프로세스의 I/O 작업(페이지 폴트)과 다른 프로세스의 실행 중첩(overlap)시키는 것은 멀티 프로그램된 시스템에서 하드웨어를 최대한 효율적으로 사용하는 방법 중 하나다.

여담: 페이지 폴트를 하드웨어로 처리하지 않는 이유

TLB 를 다뤄본 경험에서 우리는 하드웨어 설계자들이 운영체제가 하는 일을 신뢰하고 싶어하지 않는다는 것을 알았다.

어째서 운영체제에게 페이지 폴트 처리를 맡길까?

몇 가지 중요한 이유들이 있다.

첫째는 페이지 폴트로 인한 디스크 접근이 느리기 때문이다.

페이지 폴트 처리에 시간이 소요된다 하더라도, 디스크 작업 자체가 너무 느리기 때문에, 소프트웨어를 실행하는 추가 부담은 절대적으로 작다.

둘째는 페이지 폴트 처리를 위해서는 하드웨어가 스왑 공간의 구조, 디스크에 I/O 를 요청하는 방법, 그리고 그외에 하드웨어가 파악하지 못하고 있는 세부적인 것들을 모두 알고 있어야 한다.

성능과 간편성이라는 두 가지 장점으로, 페이지 폴트 처리를 운영체제가 담당하게 되었고, 하드웨어 설계자도 이것을 만족하게 되었다.

[4] 메모리에 빈 공간이 없으면?

위 설명에서, 스왑 공간으로부터 페이지를 가져오기 위한 (page-in) 여유 메모리가 충분하다는 것을 가정했다.

항상 이런 경우만 있는 것은 아니다.

메모리에 여유 공간이 없을 수도 있다.(또는 거의 다 찼을 수도 있다.)

탑재하고자 하는 새로운 페이지(들)를 위한 공간을 확보하기 위해 하나 또는 그 이상의 페이지들을 먼저 페이지 아웃(Page-out)하려고 할 수도 있다.

교체(replace) 페이지를 선택하는 것을 페이지 교체 정책(page-replacement policy)이라고 한다.

잘못된 선택은 프로그램의 성능에 큰 악영향을 미친다.

좋은 페이지 교체 정책을 만들기 위해 많은 노력들이 있었다.

잘못된 선택을 할 경우, 프로그램이 메모리 속도로 실행되는 것이 아니라, 디스크와 비슷한 속도로 동작할 수도 있기 때문이다.

현재 기술에서 볼 때, 프로그램이 10,000 또는 100,000 배 더 느리게 실행된다는 것을 의미한다.

페이지 교체 정책은 면밀히 검토되어야 한다.

다음 장에서 각종 다른 정책들에 대해서 자세히 다룬다.

현재로서는 페이지 교체 정책이라는 것이 존재한다는 사실과 그 기법이 이전에 다른 스와핑과 함께 동작한다는 점만 인지하고 넘어간다.

[5] 페이지 폴트의 처리

이제 메모리 접근이 이루어지는 전체 과정을 파악할 수 있게 되었다.

“프로그램이 메모리에서 데이터를 가져올 때 어떤 일이 발생하는가?라는 질문을 받았을 때, 다양한 상황에 대해 심도 있는 설명이 가능하게 된 것이다.

```
1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True)           // TLB 히트
4     if (CanAccess(TlbEntry.ProtectBits) == True)
5         Offset = VirtualAddress & OFFSET_MASK
6         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7         Register = \gndx{AccessMemory}(\gndx{PhysAddr})
8     else
9         RaiseException(PROTECTION_FAULT)
10 else                           // TLB 미스
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = \gndx{AccessMemory}(\gndx{PTEAddr})
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else
16         if (CanAccess(PTE.ProtectBits) == False)
17             RaiseException(PROTECTION_FAULT)
18         else if (PTE.Present == True)
19             // 하드웨어를 기반으로 한 TLB를 가정함
20             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21             RetryInstruction()
22         else if (PTE.Present == False)
23             RaiseException(PAGE_FAULT)
```

〈그림 24.2〉 페이지 오류 제어 흐름의 알고리즘(하드웨어)

```
1 PFN = FindFreePhysicalPage()
2 if (PFN == -1)                    // 비어있는 페이지 못 찾음
3     PFN = EvictPage()            // 교체 알고리즘 실행
4     \gndx{DiskRead}(\gndx{PTE.DiskAddr, pfn}) // 대기(I/O를 기다리기)
5     PTE.present = True           // 존재한다고 페이지 테이블에 갱신
6     PTE.PFN = PFN               // 비트와 변환(PFN)
7     RetryInstruction()          // 명령어 재시도
```

〈그림 24.3〉 페이지 오류 제어 흐름의 알고리즘(소프트웨어)

그림 24.2 와 그림 24.3 은 페이지 폴트 처리의 상세한 과정을 나타낸다.

첫 그림은 하드웨어를 통한 주소 변환을 나타낸다.

두 번째 그림은 페이지 폴트 발생 시 운영체제의 동작(소프트웨어)을 나타낸다.

그림 24.2 의 하드웨어 처리 과정에서, TLB 미스 발생 시, 세 가지 중요한 경우가 있다는 것을 알 수 있다.

첫째는 페이지가 존재하며 유효한 경우이다. (라인 18-21)

이 경우에도 TLB 미스 핸들러가 PTE 에서 PFN 을 가져와서 (여러 차례) 명령어를 재시도 한다(이때에는 TLB 히트가 된다.)

두 번째 경우에는 (라인 22-23)페이지가 유효하지만 존재하지 않는 경우다.

페이지 폴트 핸들러가 반드시 실행되어야 한다.

프로세스가 사용할 수 있는 제대로된 페이지이기는 하지만(유효한 경우이기 때문에) 물리 메모리에 존재하지 않기 때문이다.

세 번째, 마지막으로 페이지가 유효하지 않는 경우다.

프로그램 버그 등으로 잘못된 주소를 접근하는 경우의 처리를 나타낸다.(라인 13-14)

이 경우에는 PTE 의 다른 비트는 의미가 없다.

하드웨어는 이 무효한 접근이 운영체제의 트랩 핸들러에 의해서 처리되도록 한다.

이때 문제를 일으킨 프로세스는 종료될 수가 있다.

그림 24.3 에서 운영체제가 페이지 폴트를 처리하는 과정을 대략적으로 볼 수 있다.

먼저 운영체제는 탑재할 페이지를 위한 물리 프레임을 확보한다.

만약 여유 프레임이 없다면, 교체 알고리즘을 실행하여 메모리에서 페이지를 내보내고 여유 공간을 확보한다.

물리 프레임을 확보한 후, I/O 요청을 통해 스왑 영역에서 페이지를 읽어 온다.

마지막으로 이 느린 작업이 완료되면 운영체제는 페이지 테이블을 갱신하고 명령어를 재시도한다.

재시도를 하게 되면 TLB 미스가 발생하며, 또 한 번의 재시도를 할 때에 TLB 히트가 된다.

그래서야 하드웨어는 원하는 것을 접근할 수 있게 된다.

[6] 교체는 실제 언제 일어나는가

이제까지의 설명에서는 메모리에 여유 공간이 고갈된 후에 교체 알고리즘이 작동하는 것을 가정하였다.

이 방법은 그리 효율적이지 않다. 뿐만 아니라, 다양한 이유로 인해, 운영체제는 항상 어느 정도의 여유 메모리 공간을 확보하고 있어야 한다.

메모리에 항상 어느 정도의 여유 공간을 비워두기 위해서, 대부분의 운영체제들은 여유 공간에 관련된 최댓값(high watermark, HW)과 최솟값(low watermark, LW)을 설정하여 교체 알고리즘 작동에 활용한다.

동작 방식은 다음과 같다.

운영체제가 여유 공간의 크기가 최소값보다 작아지면 여유 공간 확보를 담당하는 백그라운드 쓰레드가 실행된다.

이 쓰레드는 여유 공간의 크기가 최댓값에 이를 때까지 페이지를 제거한다.

이 백그라운드 쓰레드는 일반적으로 스왑 데몬(swap daemon) 또는 페이지 데몬(page daemon)이라고 불린다.

충분한 여유 메모리가 확보되면 이 백그라운드 쓰레드는 슬립 모드로 들어간다.

일시에 여러 개를 교체하면 성능 개선이 가능하다.

예를 들어, 많은 시스템들은 다수의 페이지들을 클러스터(cluster)나 그룹(group)으로 묶어서 한 번에 스왑 파티션에 저장함으로써 디스크의 효율을 높인다.

디스크 관련 부분에서 좀 더 자세히 다루겠지만, 클러스터링은 디스크의 탐색(seek)과 회전 지연(rotational delay)에 대한 오버헤드를 경감시켜 성능을 상당히 높일 수 있다.

백그라운드 페이징 쓰레드를 사용하기 위해서는 그림 24.3의 과정이 약간 변경되어야 한다.

교체를 직접 수행하는 대신 알고리즘은 사용할 수 있는 페이지들이 있는지를 단순히 검사만 한다.

만약 없다면 백그라운드 페이징 쓰레드에게 여유 페이지들이 필요하다고 알려준다.

쓰레드가 페이지들을 비운 후에 원래의 쓰레드를 다시 깨워서 원하는 페이지를 불러들일 수 있도록 하며 계속 작업을 실행할 수 있도록 한다.

[1-5] 가상화: 물리 메모리 크기의 극복_정책

핵심 질문: 내보낼 페이지는 어떻게 결정하는가

운영체제는 어떻게 메모리에서 내보낼 페이지(또는 페이지들)를 결정할 수 있을까?
이 결정은 시스템의 교체 정책에 의해서 내려진다.

교체 정책은 보편 타당한 원칙들을 따르지만 코너 케이스를 피하기 위한 수정 사항들도 포함되어 있다.

가상 메모리 관리자의 입장에서 비어 있는 메모리가 많을수록 일은 쉬워진다.
페이지 폴트가 발생하면 빈페이지 리스트에서 비어 있는 페이지를 찾아서 폴트를 일으킨 페이지에게 할당하면 된다.

불행하게도 빈 메모리 공간이 거의 없으면 일이 더 복잡해진다.

그런 경우 운영체제는 메모리 압박(memory pressure)을 해소하기 위해 다른 페이지들을 강제로 페이지 아웃(page out)하여 활발히 사용 중인 페이지들을 위한 공간을 확보한다.

내보낼(evict) 페이지(또는 페이지들) 선택은 운영체제의 교체 정책(replacement policy) 안에 집약되어 있다.

과거의 시스템들은 물리 메모리의 크기가 작았기 때문에 초기 가상 메모리 시스템의 가장 중요한 역할 중 하나가 바로 이 교체 정책이었다.

[1] 캐시 관리

정책에 대해서 논하기 전에 먼저 해결하고자 하는 문제에 대해서 좀 더 상세히 설명하도록 하겠다.

시스템의 전체 페이지들 중 일부분만이 메인 메모리에 유지된다는 것을 가정하면, 메인 메모리는 시스템의 가상 메모리 페이지를 가져다 놓기 위한 캐시로 생각될 수 있다.

이 캐시를 위한 교체 정책의 목표는 캐시 미스의 횟수를 최소화하는 것이다.
즉, 디스크로부터 페이지를 가져오는 횟수를 최소로 만드는 것이다.

한편으로 캐시 히트 횟수를 최대로 하는 것도 목표라고 할 수 있다.

즉, 접근된 페이지가 메모리에 이미 존재하는 횟수를 최대로 하는 것이다.

캐시 히트와 미스의 횟수를 안다면 프로그램의 평균 메모리 접근 시간(average memory access time, AMAT)을 계산할 수 있다(컴퓨터 아키텍처가 하드웨어 캐시의 성능을 측정할 때 사용하는 미터법이다).

히트와 미스의 정보를 안다면 프로그램의 AMAT는 다음과 같은 식으로 계산할 수가 있다.

$$AMAT = (P_{hit} \times T_M) + (P_{miss} \times T_D)$$

여기서 T_M 은 메모리 접근 비용을 나타내고, T_D 는 디스크 접근 비용, P_{hit} 는 캐시에서 데이터 항목을 찾을 확률을 나타내며(히트), P_{miss} 는 캐시에서 데이터를 못찾을 확률(미스)을 나타낸다.

P_{hit} 와 P_{miss} 는 각각 0.0 에서 1.0 사이의 값을 가지며 $P_{hit} + P_{miss} = 1.0$ 을 만족한다.

예를 들어, 어떤 컴퓨터가 4KB 의 작은 메모리를 가지고 있고, 페이지의 크기는 256 바이트라고 하자.

가상 주소는 4 비트 VPN(최상위 비트)과 8 비트 오프셋(최하위 비트)의 두 부분으로 구성된다.

그러므로 이 예제에서 한 프로세스가 2^4 또는 16 개의 가상 페이지들에 접근할 수 있다.

프로세스는 가상 주소 0x00, 0x100, 0x200, 0x300, 0x400, 0x500, 0x600, 0x700, 0x800, 0x900 의 메모리를 가리키고 있다.

이 가상 주소들은 주소 공간의 첫 열 개 페이지들의 첫 번째 바이트를 나타낸다.(각 가상 주소들의 첫 16 진수 숫자가 페이지 번호를 나타낸다.)

또한, 가상 페이지 3 을 제외한 모든 페이지가 이미 메모리에 있다고 가정하자.

주어진 순서의 메모리 참조는 다음과 같은 동작을 보일 것이다.

- 히트, 히트, 히트, 미스, 히트, 히트, 히트, 히트, 히트, 히트.

히트율(hit rate)을 (메모리에 참조 대상을 찾은 백분율) 계산하면 10 개의 참조 중에서 9 개가 메모리에 있었기 때문에 90%가 된다. ($P_{hit} = 0.9$)

미스율(miss rate)은 당연히 10% ($P_{miss} = 0.1$)가 된다.

AMAT 를 계산하기 위해서 메모리를 접근하는 비용과 디스크를 접근하는 비용을 알아야한다.

메모리를 접근하는 비용(T_M)이 약 100nsec 라고 하고 디스크를 접근하는 비용(T_D)이 약 10 μ sec 라고 가정할 때 AMAT 은 $(0.9 \times 100ns) + (0.1 \times 10ms)$ 계산하게 되면 1.009 msec 혹은 약 1 μ sec 가 된다.

만약 히트율이 99.9%가 되었다면 결과는 확연히 달라진다.

즉, AMAT 가 10.1 microseconds 혹은 대략 100 배 더 빨라진다.

히트율이 100%에 가까워질수록 AMAT 는 100nsec 에 가까워진다.

이 예제에서 볼 수 있듯이 현대 시스템에서는 디스크 접근 비용이 너무 크기 때문에, 아주 작은 미스가 발생하더라도 전체적인 AMAT 에 큰 영향을 주게 된다. 컴퓨터가 디스크 속도 수준으로 느리게 실행되는 것을 방지하기 위해서는 당연히 미스를 최대한 줄여야한다. 이를 위해서는 좋은 정책을 잘 만드는 것이다.

[2] 최적 교체 정책

교체 정책의 동작 방식을 잘 이해하기 위해 최적 교체 정책(The Optimal Replacement Policy)과 비교하는 것이 좋다.

최적 정책(optimal replacement policy)은 수년 전 Belady 에 의해 개발되었다.

최적 교체 정책은 미스를 최소화한다.

Belady 는 가장 나중에 접근될 페이지를 교체하는 것이 최적이며, 가장 적은 횟수의 미스를 발생시킨다는 것을 증명하였다.

이 정책은 간단하지만 구현하기는 어려운 정책이다.

최적 기법이 직관적으로 쉽게 이해되었기 바란다.

만약 페이지 하나를 내보내야 한다면 지금부터 가장 먼 시점에 필요하게 될 페이지를 버리는 것이 좋지 않을까?

핵심은 가장 먼 시점에 필요한 페이지보다 캐시에 존재하는 다른 페이지들이 더 중요하다는 것이다.

이 주장이 참인 이유는 간단하다.

가장 먼 시점에 접근하게 될 페이지보다 다른 페이지들을 먼저 참조할 것이기 때문이다.

접근	히트/미스?	내보냄	결과적인 캐시 상태
0	미스		0
1	미스		0, 1
2	미스		0, 1, 2
0	히트		0, 1, 2
1	히트		0, 1, 2
3	미스	2	0, 1, 3
0	히트		0, 1, 3
3	히트		0, 1, 3
1	히트		0, 1, 3
2	미스	3	0, 1, 2
1	히트		0, 1, 2

간단한 예제를 통해 최적 기법의 동작을 살펴보자.

프로그램이 0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1의 순서대로 가상 페이지들을 접근한다고 하자.

캐시에 세 개의 페이지를 저장할 수 있다고 가정할 때, 그림은 최적의 기법의 동작을 보여준다.

그림에서 다음과 같은 동작을 볼 수 있다.

캐시는 처음에 비어 있는 상태로 시작하기 때문에 첫 세 번의 접근은 당연히 미스이다.

이러한 종류의 미스는 때로는 최초 시작 미스(cold-start miss) 또는 강제 미스(compulsory miss)라고 한다.

그 다음에 페이지 0과 1을 참조하면서 캐시에서 히트된다.

마지막으로 또 다른 미스를 만나게 된다(페이지 3).

그렇지만 이때는 캐시가 가득 차있기 때문에 교체 정책이 실행되어야 한다.

이때 다음과 같은 질문이 나온다.

어떤 페이지를 교체해야 할까?

최적 기법의 경우 캐시에 현재 탑재되어 있는 각 페이지들(0, 1, 2)의 미래를 살펴본다.

그러면 0은 거의 즉시 접근이 될 것이며 1은 약간 후에, 그리고 2는 가장 먼 미래에 접근이 될 것이란 것을 알 수 있다.

그러므로 최적 기법은 쉬운 선택을 하면 된다.

페이지 2를 내보내고 결과적으로 캐시에는 페이지 0, 1, 3이 남게 된다.

그 후의 세 번의 참조는 히트된다.

그 다음으로 오래 전에 내보냈던 페이지 2를 만나서 또 한 번의 미스를 경험하게 된다.

여기서 최적 기법은 다시 한 번 캐시 내의 각 페이지들(0, 1, 3)의 미래를 검사한다. 그리고 페이지 1만 아니면(바로 다음에 접근될 예정이라)어느 페이지를 내보내도 괜찮다는 것을 알게 된다.

페이지 0을 선택하는 것도 괜찮은 결정이었을 수도 있지만, 예제에서는 페이지 3을 내보냈다.

최종적으로 페이지 1이 히트되고 종료된다.

캐시 히트율을 계산할 수 있다. 캐시 히트가 6번 미스가 5번이었으므로 히트율은

$\frac{Hits}{Hits+Misses}$ 으로 계산되며 $\frac{6}{6+5}$ 또는 54.5%가 된다.

강제 미스들(즉, 해당 페이지가 처음 미스된 경우)을 제외한 히트율을 계산한다면 85.7%의 히트율을 얻는다.

불행하게도 스케줄링 정책을 만들 때도 보았지만, 미래는 일반적으로 미리 알 수 없다.

그렇기 때문에 범용 운영체제에서는 최적 기법의 구현은 불가능하다.

내보낼 페이지를 결정하기 위한 실질적이고 배포가 가능한 정책을 만들기 위해 다른 방법을 찾는 것에 집중할 것이다.

최적 기법은 비교 기준으로만 사용될 것이며, 이를 통해 “정답”에 얼마나 가까운지 알 수 있다.

[3] 간단한 정책: FIFO

초기의 많은 시스템들은 최적의 방법에 도달하려는 복잡한 시도를 피하고 대신 매우 간단한 교체 정책을 채용하였다.

예를 들면, 일부 시스템에서는 FIFO(먼저 들어온 것이 먼저 나간다. 선입선출) 교체 방식을 사용하였다.

접근	히트/미스?	내보냄	결과적인 캐시 상태
0	미스		선입 0
1	미스		선입 0, 1
2	미스		선입 0, 1, 2
0	히트		선입 0, 1, 2
1	히트		선입 0, 1, 2
3	미스	0	선입 1, 2, 3
0	미스	1	선입 2, 3, 0
3	히트		선입 2, 3, 0
1	미스	2	선입 3, 0, 1
2	미스	3	선입 0, 1, 2
1	히트		선입 0, 1, 2

〈그림 25.2〉 FIFO 정책의 흐름

이 방법은 시스템에 페이지가 들어오면 큐에 삽입되고, 교체를 해야할 경우 큐의 테일에 있는 페이지가(“먼저 들어온” 페이지) 내보내진다.

FIFO 는 매우 구현하기 쉽다는 장점을 가진다.

예제로 사용한 메모리 참조의 흐름에서 FIFO 가 어떻게 동작하는지 살펴보자.

처음에는 마찬가지로 페이지 0, 1 과 2 에 대한 세 개의 강제 미스로 시작하고, 0 과 1 이 다음에 히트된다.

다음으로 페이지 3 이 참조되지만 미스를 일으킨다.

FIFO 를 사용할 때 교체 결정은 쉽다.

순서상 “첫 번째”로 들어온 페이지인 0 을 선택하면 된다.

불행하게도 다음 접근은 페이지 0 이라서 또 다른 미스를 만들어내고

교체가(페이지 1) 필요하다.

그 후에는 페이지 3 에서 히트가 되지만 1 과 2 는 미스가 되며, 끝으로 3 은 히트된다.

최적의 경우와 비교하면 FIFO 는 눈에 띄게 성능이 안좋다.

히트율은 36.4%가 된다.(또는 강제 미스를 제외하면 57.1%가 된다.)

FIFO 는 블록들의 중요도를 판단할 수가 없다.

페이지 0 이 여러 차례 접근이 되었더라도 단순히 메모리에 먼저 들어왔다는 이유로 FIFO 는 페이지 0 을 내보낸다.

[4] 또 다른 간단한 정책: 무작위 선택

또 다른 유사한 교체 정책은 무작위 방식이다.

이 방식은 메모리 압박이 있을 때 페이지를 무작위로 선택하여 교체한다.

무작위 선택 방식은 FIFO 와 유사한 성질을 가지고 있다.

구현하기 쉽지만 내보낼 블록을 제대로 선택하려는 노력을 하지 않는다.

앞선 예제에서 사용한 메모리 참조를 사용하여 무작위 선택 정책이 어떻게 동작하는지 살펴보자.

물론 무작위 선택 정책은 선택할 때 얼마나 운이 좋은지(또는 운이 나쁜지)에 전적으로 의존한다.

위에서 사용한 참조 열에서는 무작위 선택 방식이 FIFO 보다는 약간 더 좋은 성능을 보이며 최적의 방법보다는 약간 나쁜 성능을 보인다.

무작위 선택 방식의 일반적인 성능을 알아보기 위해 실험을 수천 번 반복해 볼 수 있다.

무작위 선택 방식의 동작은 그때그때에 따라 달라진다.

[5] 과거 정보의 사용: LRU

불행하게도 FIFO 또는 무작위 선택 방식처럼 단순한 정책들은 중요한 페이지들을 혹은 바로 다시 참조하게 될 것들을 내보낼 수 있다는 비슷한 문제를 겪게 된다.

FIFO는 먼저 가져온 페이지를 먼저 내보낸다.

그 페이지에 중요한 코드 또는 자료 구조가 들어 있고, 곧 다시 필요하다 해도, 먼저 들어온 페이지라면 내보낸다. FIFO와 무작위 선택 방식 그리고 그와 유사한 정책들은 최적 기법의 성능을 따라갈 수가 없기 때문에 좀 더 정교한 방식이 필요하다.

스케줄링 정책에서와 같이 미래에 대한 예측을 위해서 과거 사용 이력을 활용해보자.

예를 들어 어떤 프로그램이 가까운 과거에 한 페이지를 접근했다면 가까운 미래에 그 페이지를 다시 접근하게 될 확률이 높다.

페이지 교체 정책이 활용할 수 있는 과거 정보 중 하나는 빈도수(frequency)이다. 만약 한 페이지가 여러 차례 접근되었다면, 분명히 어떤 가치가 있기 때문에 교체되면 안될 것이다.

좀 더 자주 사용되는 페이지의 특징은 접근의 최근성(recency)이다.

더 최근에 접근된 페이지일수록 가까운 미래에 접근될 확률이 높을 것이다.

이러한 류의 정책은 지역성의 원칙(principle of locality)이라고 부르는 특성에 기반을 둔다.

이 원칙은 프로그램의 행동 양식을 관찰하여 얻은 결과이다.

이 원칙이 말하는 것은 단순하다.

프로그램들은 특정 코드들과 자료 구조를 상당히 빈번하게 접근하는 경향이 있다는 것이다.

코드의 예로는 반복문 코드를 들 수 있으며, 자료 구조로는 그 반복문에 의해 접근되는 배열을 예로 들 수 있다.

과거의 현상을 보고 어떤 페이지들이 중요한지 판단하고, 내보낼 페이지를 선택할 때 중요한 페이지들은 메모리에 보존하는 것이다.

그리하여 과거 이력에 기반한 교체 알고리즘 부류가 탄생하게 되었다.

Least-Frequently-Used(LFU) 정책은 가장 적은 빈도로 사용된 페이지를 교체한다.

Least-Recently-Used(LRU) 정책은 가장 오래 전에 사용하였던 페이지를 교체한다.

이러한 알고리즘들은 기억하기 쉽다.

이름을 알면 정확하게 그 알고리즘이 어떻게 동작하는지 알 수 있다.

이름이 가지는 탁월한 성질이 아닐 수 없다.

접근	히트/미스?	내보냄	결과적인 캐시 상태
0	미스		LRU 0
1	미스		LRU 0, 1
2	미스		LRU 0, 1, 2
0	히트		LRU 1, 2, 0
1	히트		LRU 2, 0, 1
3	미스	2	LRU 0, 1, 3
0	히트		LRU 1, 3, 0
3	히트		LRU 1, 0, 3
1	히트		LRU 0, 3, 1
2	미스	0	LRU 3, 1, 2
1	히트		LRU 3, 2, 1

〈그림 25.5〉 LRU 정책의 흐름

LRU 방식을 좀 더 이해하기 위해서 앞서 사용한 참조 패턴에서 예제가 어떻게 동작하는지 알아보자.

그림에 그 결과가 있다.

그림을 보면 상태를 유지하지 않는 랜덤이나 FIFO 정책에 비해 LRU가 과거 정보를 사용하여 더 좋은 성능을 보임을 알 수 있다.

이 예제에서 LRU는 처음으로 페이지를 교체해야 할 때가 되었을 때 페이지 2를 내보낸다.

왜냐하면 0과 1이 최근에 사용되었기 때문이다.

그 후에는 페이지 0을 교체한다.

1과 3이 더 최근에 접근되었기 때문이다.

두 경우 모두 LRU는 과거 정보를 활용하였고, 결과적으로 이후의 참조들에서 히트가 발생하였다는 것을 보면 결정이 옳았다는 것을 알 수 있다.

이 예제에서 LRU가 최적 기법과 같은 정도 수준의 성능을 얻을 수 있는 최고의 결과를 보여주고 있다.

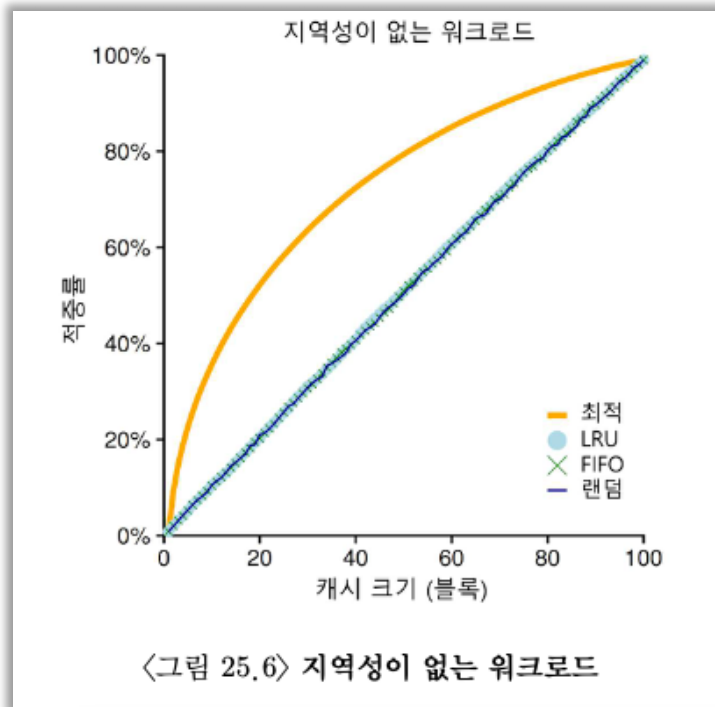
이와 반대되는 Most-Frequently-Used(MFU)와 Most-Recently-Use(MRU)와 같은 알고리즘도 존재하지만 대부분의 경우(모든 경우는 아님), 이러한 정책들은 잘 동작하지 않는다.

대부분의 프로그램들에서 관찰되는 지역성 접근 특성을 사용하지 않고 오히려 무시하기 때문이다.

[6] 워크로드에 따른 성능 비교

지금까지 살펴본 정책들 동작을 더 잘 이해하기 위해 몇 가지 예제들을 더 살펴보자. 이번에는 짧은 흐름 대신 좀 더 복잡한 워크로드를 살펴볼 것이다.

하지만 이러한 워크로드조차도 상당히 간단화된 것이기 때문에 제대로 된 연구를 위해서는 응용프로그램의 트레이스 자료를 사용해야 할 것이다.



첫 번째 살펴볼 워크로드에서는 지역성이 없다.

그 말은 접근되는 페이지들의 집합에서 페이지가 무작위적으로 참조된다는 것을 의미한다.

이 예제에서는 100 개의 페이지들이 일정 시간 동안 계속 접근하는 워크로드를 사용한다.

접근되는 페이지는 무작위적으로 선택되며 페이지들이 총 10,000 번 접근된다.

실험에서 캐시의 크기를 매우 작은 것부터(한 페이지) 모든 페이지들을 담을 수 있을 정도의 크기까지(100 페이지) 증가시켰으며, 이를 통해 각 정책이 캐시 크기에 따라 어떻게 동작하는지 살펴보았다.

그림에서는 최적의 방법과 LRU, 랜덤 그리고 FIFO 방식을 사용하였을 때의 실험 결과를 나탄낸다.

그림의 y 축은 각 정책이 달성한 히트율을 나타내며 x 축은 앞서 설명한 캐시 크기의 변화를 나탄낸다.

그림에서 몇 가지 결론을 얻을 수 있다.

먼저, 워크로드에 지역성이 없다면 어느 정책을 사용하든 상관이 없다.

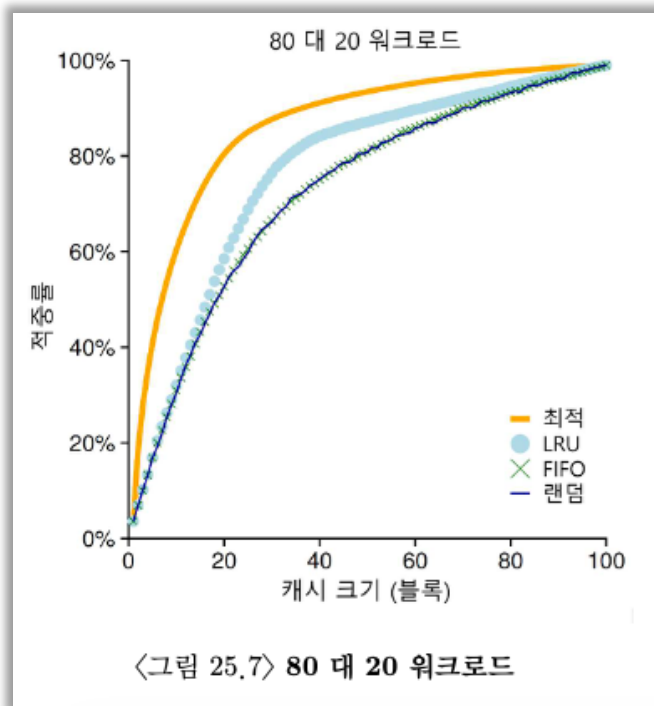
LUR 와 FIFO 그리고 무작위 선택 정책은 모두 동일한 성능을 보이며, 히트율은 정확히 캐시의 크기에 의해서 결정된다.

두 번째, 캐시가 충분히 커서 모든 워크로드를 다 포함할 수 있다면 역시 어느 정책을 사용하든지 상관 없다.

참조되는 모든 블럭들이 캐시에 들어갈 수 있으면 모든 정책들은(무작위 선택마저도) 히트율이 100%에 도달한다.

마지막으로, 최적 기법이 구현 가능한 기타 정책들보다 눈에 띄게 더 좋은 성능을 보인다.

미래를 알 수 있다면 교체 작업을 월등히 잘할 수 있다.



다음으로 살펴볼 워크로드는 “80 대 20” 워크로드라고 부른다.

20%의 페이지들에서 (“인기 있는” 페이지) 80%의 참조가 발생하고 나머지 80%의 페이지들에 대해서 20%의 참조만 (“비인기” 페이지) 발생한다.

이 워크로드에서는 마찬가지로 총 100 개의 페이지들이 있다.

“인기 있는” 페이지들에 대한 참조가 실험 시간의 대부분을 차지한다.

그림에는 각 정책들이 워크로드에서 어떻게 동작하는지 보여주고 있다.

그림에서 볼 수 있듯이 랜덤과 FIFO 정책이 상당히 좋은 성능을 보이지만, 인기 있는 페이지들을 캐시에 더 오래두는 경향이 있는 LRU가 더 좋은 성능을 보인다.

인기 있는 페이지들이 과거에 빈번하게 참조되었기 때문에 그 페이지들은 가까운 미래에 다시 참조되는 경향이 있기 때문이다.

최적 기법은 여전히 더 좋은 성능을 보이고 있으며, 이는 LRU의 과거 정보가 완벽하지는 않다는 것을 보여준다.

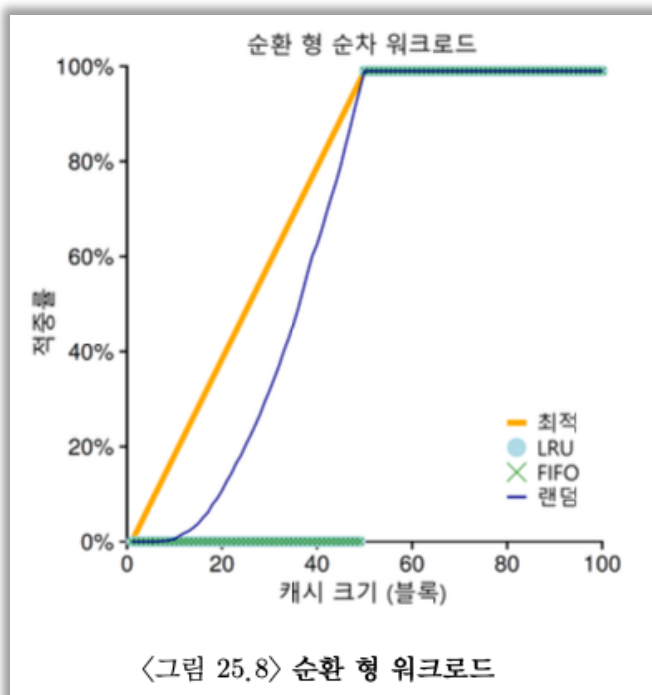
이런 의문이 들 수 있다.

무작위 선택 방식과 FIFO 방식을 개선하여 만든 LRU가 그렇게 대단한 것인가?

그 대답은 항상 그렇듯이 “상황에 따라 다르다”이다.

만약 각 미스로 인해서 매우 비싼 값을 치러야 한다면(드물게 일어나는 일이 아니다) 히트율이 약간만 증가하더라도(미스율이 줄어들면) 성능에 큰 차이를 만들어 낼 수 있다.

만약 미스로 인한 영향이 그렇게 크지 않다면, LRU로 얻을 수 있는 장점은 당연히 그렇게 중요하지 않게 된다.



마지막으로 하나의 워크로드를 더 살펴보자.

이 워크로드를 “순차 반복” 워크로드라고 부른다.

이 워크로드는 50 개의 페이지들을 순차적으로 참조한다.

0 번 페이지를 참조하고 1 번을 참조하고 ... 49 번째의 페이지를 참조한 후에 다시 처음으로 돌아가서 그 접근 순서를 반복한다.

50 개의 개별적인 페이지들을 총 10,000 번 접근한다.

그림에서 보이는 마지막 도표는 각 정책들이 워크로드에서 어떤 성능을 보이는지 나타낸다.

여러 응용 프로그램에서 흔하게 볼 수 있는 이 워크로드는(데이터베이스와 같은 상업용 응용 프로그램들을 포함하여) LRU와 FIFO 정책에서 가장 안좋은 성능을 보인다.

순차 반복 워크로드에서 이 알고리즘들은 오래된 페이지들을 내보낸다.

불행하게도 워크로드의 반복적인 특성으로 인해서 오래된 페이지들은 정책들이 캐시에 유지하려고 선택한 페이지들보다 먼저 접근된다.

실제로, 캐시의 크기가 49 라 할지라도 50 개의 페이지들을 순차 반복하는 워크로드에서는 캐시 히트율이 0%가 된다.

흥미롭게도 무작위 선택 정책은 최적의 경우에 못 미치기는 하지만 눈에 띄게 좋은 성능을 보인다.

무작위 선택 정책의 히트율은 최소한 0%는 아니다.

이렇게 무작위 선택 정책은 몇 가지 좋은 특성을 가지고 있다는 것을 알 수 있다. 그 중 한가지는 이상한 코너 케이스가 발생하지 않는다는 것이다.

[7] 과거 이력 기반 알고리즘의 구현

살펴본 것처럼 중요한 페이지들을 내보낼 수도 있는 단순한 FIFO와 무작위 선택 정책들 보다 LRU와 같은 알고리즘이 더 좋은 성능을 보인다.

하지만 과거 정보에 기반을 둔 정책은 새로운 문제점이 있다.

이런 정책을 어떻게 구현할 것인가?

LRU를 예로 들어보자.

이를 완벽하게 구현하기 위해서 많은 작업을 해야 한다.

특히, 각 페이지 접근마다(즉, 명령어 탑재나 로드 또는 저장하려는 각 메모리 접근) 해당 페이지가 리스트에 가장 앞으로 이동하도록(즉, MRU 측으로) 자료 구조를 갱신해야 한다.

FIFO 방식과 비교해보자. FIFO 리스트는 어떤 페이지의 제거(가장 먼저 들어온 페이지를 제거하기) 또는 새로운 페이지의 삽입 시에만(가장 나중에 들어온 쪽으로) 접근된다.

LRU에서는 어떤 페이지가 가장 최근에 또는 가장 오래전에 사용되었는지를 관리하기 위해서 모든 메모리 참조 정보를 기록해야 한다.

세심한 주의 없이 정보를 기록하면 성능이 크게 떨어질 수 있다.

이 작업을 좀 더 효율적으로 하는 방법은 약간의 하드웨어 지원을 받는 것이다.

예를 들어, 페이지 접근이 있을 때마다 하드웨어가 메모리의 시간 필드를 갱신하도록 할 수 있다.(예를 들어, 이 정보를 각 프로세스의 페이지 테이블에 포함시키거나, 물리 페이지마다 한 항목씩 할당된 배열로 보관할 수도 있다).

이렇게 하여 페이지가 접근되면 하드웨어가 시간 필드를 현재 시간으로 설정한다. 페이지 교체 시에 운영체제는 가장 오래 전에 사용된 페이지를 찾기 위해 시간 필드를 검사한다.

시스템의 페이지 수가 증가하면 페이지들의 시간 정보 배열을 검사하여 가장 오래 전에 사용된 페이지를 찾는 것은 매우 고비용의 연산이 된다.

4GB 의 메모리를 가지는 현대의 기기에서 4KB 페이지들로 나눈 것을 상상해 보라.

그런 기계는 백만 개의 페이지들을 가지고 있다.

LRU 정책의 교체 대상 페이지를 찾는 데 현대의 CPU 속도라 할지라도 오랜 시간이 걸리게 된다.

다음과 같은 질문이 떠오른다.

가장 오래된 페이지를 꼭 찾아야만 할까?

대신 비슷하게 오래된 페이지를 찾아도 되지 않을까?

[8] LRU 정책 근사하기

LRU 정책에 가까운 구현이 가능한가라는 질문에 대한 답은 “예”이다.

연산량이라는 관점에서 볼 때, LRU 를 “근사”하는 식으로 만들면 구현이 훨씬 쉬워진다.

실제로 현대의 많은 시스템이 이런 방식을 택하고 있다.

이 개념에는 use bit(때로는 reference bit 라고도 불린다)라고 하는 약간의 하드웨어 지원이 필요하다.

이 기법은 페이지징을 최초로 적용한 Atlas one-level store 시스템에 처음으로 구현되었다.

시스템의 각 페이지마다 하나의 use bit 가 있으며, 이 use bit 는 메모리 어딘가에 존재한다.(구현에 따라 프로세스마다 가지고 있는 페이지 테이블에 있을 수도 있고 또는 어딘가에 배열의 형태로 존재할 수도 있다.)

페이지가 참조될 때마다(즉, 읽히거나 기록되면) 하드웨어에 의해서 use bit 가 1 로 설정된다.

하드웨어는 이 비트를 절대로 지우지 않는다.(즉 0 으로 설정하지 않는다)

0 으로 바꾸는 것은 운영체제의 몫이다.

운영체제는 LRU 에 가깝게 구현하기 위해서 use bit 를 어떻게 활용할까?
여러 가지 방법이 있을 수 있지만, 간단한 활용법이 시계 알고리즘(clock algorithm)에서 제시되었다.
시스템의 모든 페이지들이 환형 리스트를 구현한다고 가정하자.

시계 바늘(clock hand)이 특정 페이지를 가리킨다고 해보자.(어떤 것이든 상관없다)

페이지를 교체해야 할 때 운영체제는 현재 바늘이 가리키고 있는 페이지 P 의 use bit 가 1 인지 0 인지 검사한다.

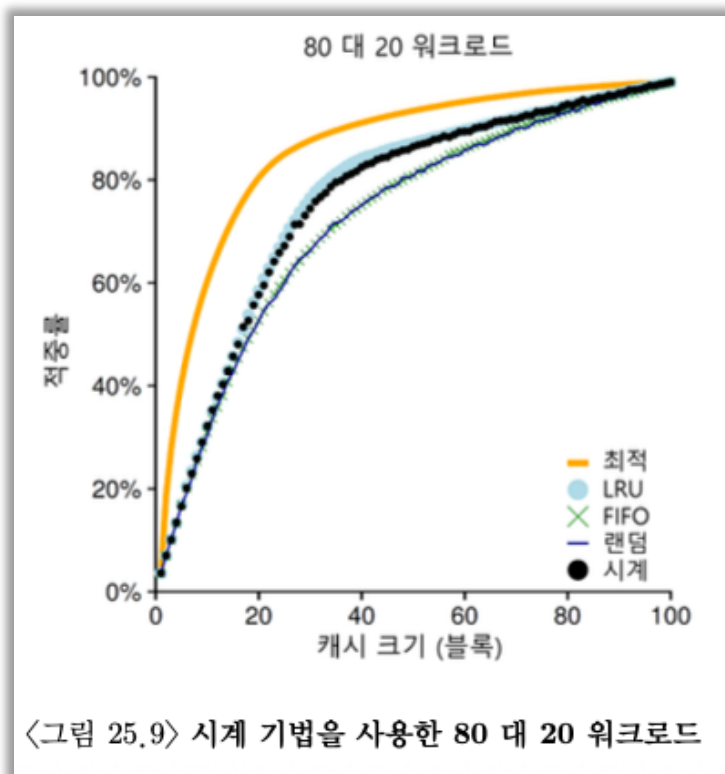
만약 1 이라면 페이지 P 는 최근에 사용되었으며 바람직한 교체 대상이 아니라는 것을 뜻한다.

P 의 use bit 는 0 으로 설정되고(지워짐) 시계 바늘은 다음 페이지 P + 1 로 이동한다.
알고리즘은 use bit 가 0 으로 설정되어 있는, 즉 최근에 사용된 적이 없는, 페이지 찾을 때까지 반복된다.(최악의 경우에는 모든 페이지들이 사용된 적이 있어서 모든 페이지들을 모두 탐색하면서 use bit 를 전부 0 으로 설정해야 할 수도 있다)

이 방법 외에도 use bit 를 사용하여 LRU 를 근사하는 다른 방법들이 존재한다.

주기적으로 use bit 를 지우고, 교체 대상 페이지 선택을 위해 use bit 가 1 과 0 인 페이지를 구분할 수 있으면 된다.

Corbato 의 알고리즘은 미사용 페이지를 찾기 위해 매번 모든 메모리를 검사하지 않아도 되는 좋은 특성을 가진 하나의 초기 방법일 뿐이다.



변형된 시계 알고리즘의 동작이 그림에 나와 있다.

이 변형 방식은 교체할 때 페이지들을 랜덤하게 검사한다.

reference bit 가 1로 설정되어 있는 페이지를 만나게 되면 비트를 지운다.(즉, 0으로 설정)

reference bit 가 0으로 설정되어 있는 페이지를 만나면 그 페이지를 교체 대상으로 선정한다.

완벽한 LRU 만큼 좋은 성능을 보이지는 않지만 과거 정보를 고려하지 않는 다른 기법들에 비해서는 성능이 더 좋다.

[9] 갱신된 페이지(Dirty Page)의 고려

많은 곳에서 실제 사용되고 있는 시계 알고리즘에 대한 추가 개선 사항을 논의하고자 한다.

이 사항은 Corbato 가 처음 제안하였으며, 운영체제가 교체 대상을 선택할 때 메모리에 탑재된 이후에 변경되었는지를 추가적으로 고려하는 것이다.

이것이 피롱한 이유는 다음과 같다.

만약에 어떤 페이지가 변경(modified)가 되어 더티(dirty) 상태가 되었다면, 그 페이지를 내보내기 위해서는 디스크에 변경 내용을 기록해야 하기 때문에 비싼 비용을 지불해야 한다.

만약 변경되지 않았다면(그러므로 깨끗한 상태(clean)), 내보낼 때 추가 비용은 없다. 해당 페이지 프레임은 추가적인 I/O 없이 다른 용도로 재사용될 수 있다. 때문에 어떤 VM 시스템들은 더티 페이지 대신 깨끗한 페이지를 내보내는 것을 선호한다.

이와 같은 동작을 지원하기 위해서 하드웨어는 modified bit(더티 비트라고도 불림)를 포함해야 한다.

페이지가 변경될 때마다 이 비트가 1로 설정되므로 페이지 교체 알고리즘에서 이를 고려하여 교체 대상을 선택한다.

예를 들어, 시계 알고리즘은 교체 대상을 선택할 때 사용되지 않은 상태이고 깨끗한, 두 조건을 모두 만족하는 페이지를 먼저 찾도록 수정된다.

이러한 조건을 만족시키는 페이지를 찾는 데 실패하면, 수정되었지만, 한동안 사용되지 않았던 페이지를 찾는다.

[10] 다른 VM 정책들

페이지 교체 정책만이 VM 시스템이 채택하는 유일한 정책은 아니다(그 중에서 가장 중요한 것이기는 하다).

예를 들어, 운영체제는 언제 페이지를 메모리로 불러들일지 결정해야 한다.

이 정책은 운영체제에게 몇 가지 다른 옵션을 제공한다.

이 정책은 Denning 이 불렀던 것처럼 때로는 페이지 선택(page selection) 정책이라고 불린다.

운영체제는 대부분의 페이지를 읽어 들일 때 요구 페이징(demand paging) 정책을 사용한다.

이 정책은 말 그대로 “요청된 후 즉시”, 즉 페이지가 실제로 접근될 때 운영체제가 해당 페이지를 메모리로 읽어 들인다.

운영체제는 어떤 페이지가 곧 사용될 것이라는 것을 대략 예상할 수 있기 때문에 미리 메모리로 읽어 들일 수도 있다.

이와 같은 동작을 선반입(prefetching)이라고 하며 성공할 확률이 충분히 높을 때에만 해야 한다.

예를 들어, 어떤 시스템은 코드 페이지 P가 메모리에 탑재되면 P+1이 접근될 확률이 높기 때문에 P가 탑재될 때 P+1도 함께 탑재되어야 한다고 가정할 수 있을 것이다.

또 다른 정책은 운영체제가 변경된 페이지를 디스크에 반영하는 데 관련된 방식이다. 한 번에 한 페이지씩 디스크에 쓸 수 있지만, 많은 시스템은 기록해야 할 페이지들을 메모리에 모은 후, 한 번에(더 효율적으로) 디스크에 기록한다.

이와 같은 동작을 클러스터링(clustering) 또는 단순히 쓰기 모으기(grouping of writes)라고 부르며 효과적인 동작 방식이다.

왜냐하면 디스크 드라이브는 여러 개의 작은 크기의 쓰기 요청을 처리하는 것보다 하나의 큰 쓰기 요청을 더 효율적으로 처리할 수 있는 특성을 갖고 있기 때문이다.

[11] 쓰래싱(Thrashing)

마치기 전에 마지막으로 한 가지 질문에 대해 다루고자 한다.

메모리 사용 요구가 감당할 수 없을 만큼 많고 실행 중인 프로세스가 요구하는 메모리가 가용 물리 메모리 크기를 초과하는 경우에 운영체제는 어떻게 해야하는가?

이런 경우 시스템은 끊임없이 페이징을 할 수밖에 없고, 이와 같은 상황을 쓰래싱이라고 부른다.

몇몇 초기 운영체제들은 쓰래싱이 발생했을 때, 이의 발견과 해결을 위한 상당히 정교한 기법들을 가지고 있었다.

예를 들면, 다수의 프로세스가 존재할 때, 일부 프로세스의 실행을 중지시킨다. 실행되는 프로세스의 수를 줄여서 나머지 프로세스를 모두 메모리에 탑재하여 실행하기 위해서이다.

워킹 셋(working set)이란 프로세스가 실행 중에 일정 시간 동안 사용하는 페이지들의 집합이다.

일반적으로 진입 제어(admission control)라고 알려져 있는 이 방법은 많은 일들을 영성하게 하는 것보다 더 적은 일을 제대로 하는 것이 나을 때가 있다고 말한다. 현대 컴퓨터 시스템에서 뿐만 아니라 실제 삶에서도 종종 부딪치는 상황이기도 하다.

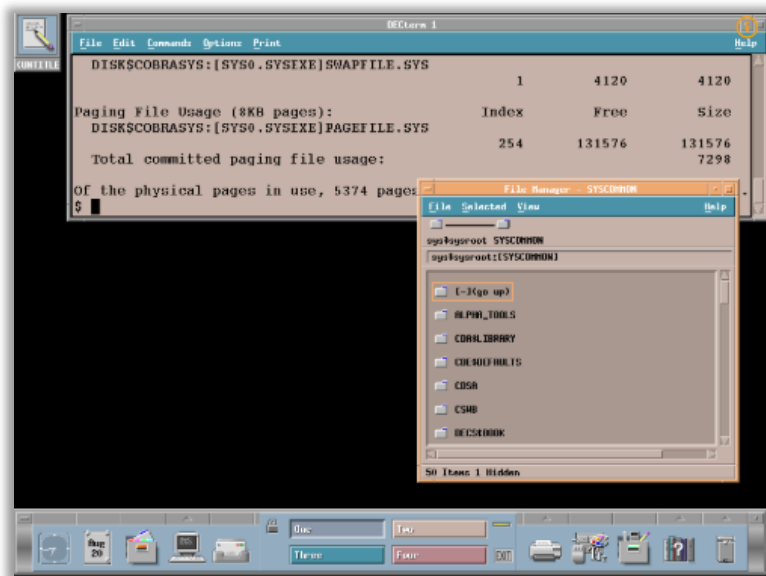
일부 최신 시스템들은 메모리 과부하에 대해서 좀 더 과감한 조치를 취하기도 한다. 예를 들면, 일부 버전의 Linux 는 메모리 요구가 초과되면 메모리 부족 킬러(out-of-memory killer)를 실행시킨다.

이 데몬은 많은 메모리를 요구하는 프로세스를 골라 죽이는, 그다지 교묘하지 않은 방식으로 메모리 요구를 줄인다.

메모리 요구량을 줄이는 데는 성공적일지 몰라도, 이 방법은 문제가 될 수 있다.

예를 들어, 만약 X 서버를 죽이게 되면 화면이 필요한 모든 응용 프로그램들을 쓸모 없게 만든다.

[1-7] 가상화: VAX/VMS 가상 메모리 시스템



가상 메모리에 대한 학습을 마치기 전에, VAX/VMS 운영체제의 잘 만들어진 가상 메모리 관리자에 대해 상세하게 살펴보자.

여기서는 이전의 장에서 살펴보았던 몇 가지 개념들이 완전한 메모리 관리자 내에 어떤 식으로 적용되었는지 설명할 것이다.

[1] 배경: 범용 시스템의 딜레마

VAX-11 미니 컴퓨터 구조는 1970년대 말 Digital Equipment Corporation(DEC)에 의해 설계되었다.

이 시스템은 저사양부터 고성능 장비까지 다양한 환경에서 동작해야 했으며, 이로 인해 “일반론의 저주”라는 문제에 직면하였다.

- 다양한 환경 지원 필요
 - 특정 환경 최적화 어려움
 - 성능 저하 가능성

이 문제를 해결하기 위해 VMS는 하드웨어의 한계를 소프트웨어로 보완하는 전략을 채택하였다.

[2] 메모리 관리 하드웨어

[2-1] VAX-11 메모리 구조 정리

VAX-11 시스템은 페이징과 세그멘테이션을 결합한 하이브리드 메모리 구조를 사용한다.

이 시스템에서는 32 비트 가상 주소 공간을 512 바이트 크기의 페이지 단위로 나누어 관리한다.

가상 주소는 다음과 같다.

- 상위 23 비트: VPN(가상 페이지 번호)
- 하위 9 비트: 오프셋

또한 VPN의 상위 2 비트는 세그먼트 구분(P0, P1, S)에 사용된다.

[2-2] 주소 공간 구조

VAX-11의 가상 주소 공간은 크게 세 가지 영역으로 나뉜다.

P0 영역(프로세스 영역 - 하위)

- 사용자 프로그램 코드 + 힙(heap)
- 힙은 주소가 증가하는 방향으로 확장

P1 영역(프로세스 영역 - 상위)

- 스택(stack)
- 스택은 주소가 감소하는 방향으로 확장

S 영역(시스템 영역)

- 운영체제 코드 및 데이터 저장
- 모든 프로세스가 공유

[낮은 주소]

p0: 코드 + 데이터 + 힙 (↑ 증가)

p1: 스택 (↓ 증가)

s: 커널 영역 (운영체제)

[높은 주소]

[2-3] 핵심 문제: 페이지 테이블 크기

VAX 는 페이지 크기가 512B 로 매우 작기 때문에

- 페이지 개수가 많아지고
- 페이지 테이블 크기가 매우 커지는 문제가 발생한다.

[2-4-1] 해결 방법 1: 세그먼트별 페이지 테이블

운영체제는 이를 해결하기 위해 다음과 같은 구조를 사용한다.

- 각 프로세스는 2 개의 페이지 테이블(P0, P1)을 가진다.
 - P0 용 페이지 테이블
 - P1 용 페이지 테이블

효과: 메모리 절약

- 사용되지 않는 중간 영역(힙과 스택 사이)에 대해 페이지 테이블을 만들 필요 없음

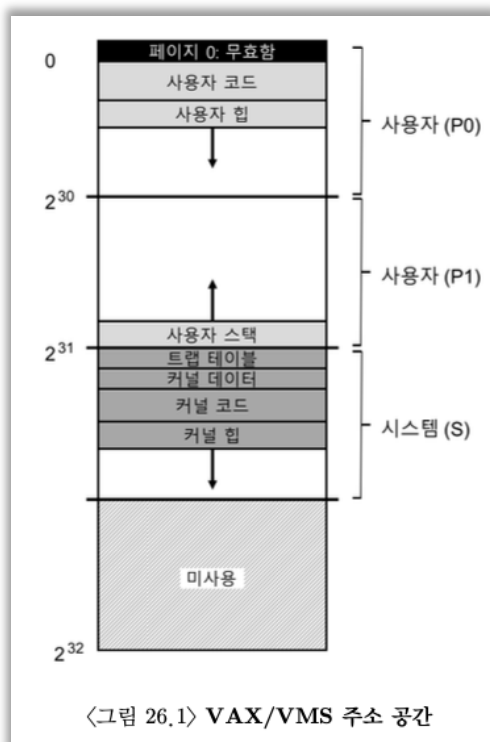
[2-4-2] Base/Bound 의 역할

각 세그먼트(P0, P1)에 대해

- Base 레지스터: 페이지 테이블 시작 주소
- Bound 레지스터: 페이지 테이블 크기

이 범위 안에서만 페이지 테이블 접근 가능

- 보호 + 범위 체크



[2-5-1] 해결 방법 2: 페이지 테이블을 커널 메모리에 저장

추가적으로 운영체제는 페이지 테이블 자체를 커널의 가상 메모리(S 영역)에 배치한다.

의미

- 페이지 테이블도 일반 데이터처럼 관리됨
- 필요 없으면 디스크로 스왑 가능

효과

- 물리 메모리 사용량 감소

[2-5-2] 단점: 주소 변환이 복잡해진다.

이 구조에서는 주소 변환이 더 복잡해진다.

일반적인 경우:

가상 주소 → 페이지 테이블 → 물리 주소

VAX 구조

- 가상 주소 → (페이지 테이블 찾기)
- (그 페이지 테이블도 가상 주소일 수 있음)
- 다시 변환
- 최종 물리 주소

즉, 페이지 테이블을 찾기 위해 또 주소 변환이 필요함

[2-5-3] 해결: TLB

이 복잡한 과정은 실제로는 TLB가 캐싱하여 해결한다.

- 복잡한 변환 과정 -> 대부분 생략
- 성능 유지 가능

[3] 실제 주소 공간 설계의 특징

[3-1] 널 포인터 보호(Null Pointer Protection)

VMS에서는 가상 주소 0번 페이지를 의도적으로 '접근 불가능' 상태로 설정한다.

이유는 이후에 설명한다.

일반적으로 프로그램에서 널 포인터는 다음과 같이 사용된다.

```
int *p = NULL; // p = 0
*p = 10;       // 잘못된 접근
```

이때 발생하는 과정은 다음과 같다.

1. CPU가 가상 주소 0에 접근 시도
2. TLB에서 해당 주소 탐색 → TLB miss
3. 페이지 테이블 조회
4. VPN 0에 해당하는 엔트리가 invalid(무효) 상태임을 확인
5. 하드웨어가 예외 발생 → 운영체제로 트랩 전달
6. 운영체제가 해당 프로세스를 종료(또는 SIGSEGV 전달)

결과

- 잘못된 메모리 접근 즉시 감지
- 프로그램 오류를 빠르게 발견 가능

의미

- 디버깅을 쉽게 하기 위한 의도적인 설계

즉, 단순한 보호가 아니라 개발자를 위한 안전장치 역할까지 수행

널 포인터 보호는 0번 주소를 막아 잘못된 메모리 접근을 즉시 발견하게 하는 디버깅 치환적 설계이다.

[3-2] 커널의 공유 구조(Kernel Mapping in User Space)

1. 주소 공간 개념

프로그램이 실행되면 운영체제(OS)는 각 프로세스에게 가상의 메모리 지도(주소 공간, Address Space)를 하나씩 제공한다.

이 주소 공간은 실제 물리 메모리가 아니라,
프로세스가 “자기만의 메모리를 가진 것처럼 보이게 하는 논리적 공간”이다.

2. 주소 공간 구조

각 프로세스의 주소 공간은 다음과 같이 구성된다.

[사용자 영역]

- 프로그램 코드 (text)
- 데이터 / 힙 (heap)
- 스택 (stack)

[커널 영역]

- 운영체제 코드
- 파일 시스템
- 드라이버
- 시스템 자원 관리

3. VMS 의 핵심 구조(S 영역)

VMS 에서는 이 주소 공간을 다음처럼 나눈다.

P0 : 사용자 코드 + 힙

P1 : 스택

S : 커널 영역

- S 영역(커널 영역)은 모든 프로세스에서 동일하게 존재하며 공유한다.

4. 동작 방식(context switch)

프로세스 전환이 발생할 때:

- P0, P1 -> 현재 프로세스에 맞게 변경됨
- S 영역 -> 변경되지 않음(항상 동일)

즉, 프로세스는 서로 다른 사용자 공간을 가지지만, 커널 공간은 모두 공유한다.

5. 왜 이렇게 설계했는가?

이 구조의 목적은 시스템을 단순하고 효율적으로 만들기 위함이다.

예를 들어, 시스템 호출이 발생할 때:

```
write(fd, buffer, size);
```

일반적인 구조에서는

- 사용자 공간 -> 커널 공간으로 데이터 복사 필요
- 추가적인 주소 변환 과정 필요

하지만 VMS 구조에서는

- 커널이 모든 프로세스의 주소 공간에 포함되어 있음
- 따라서 해당 데이터를 추가 복사 없이 직접 접근 가능

6. 효과

이 설계로 인해 다음 효과가 발생한다.

- 시스템 콜 처리 속도 향상
- 데이터 복사 비용 감소
- 커널 구조 단순화
- 커널이 마치 “공용 라이브러리”처럼 동작

7. 만약 커널을 분리했다면

커널이 별도의 주소 공간에 존재한다면

- 사용자 -> 커널 데이터 전달 시 복사 필요
- 주소 변환 과정 추가
- 시스템 호출 비용 증가
- 구조 복잡화

[3-3] 보호 메커니즘(Protection Mechanism)

커널을 사용자 주소 공간에 포함시키면 생기는 문제

- 사용자가 커널 메모리에 접근하면 어떻게 막을 것인가?

해결방법: Protection Bits

각 페이지 테이블 엔트리(PTE)에는 보호 비트가 존재한다.

- 읽기(Read) 가능 여부
- 쓰기(Write) 가능 여부
- 실행(Execute) 가능 여부
- 접근 가능한 권한 수준(User/Kernel)

동작 방식

1. 사용자 프로그램이 커널 영역 접근 시도
2. 하드웨어가 PTE 의 보호 비트 확인
3. 권한 부족 확인
4. 예외 발생 -> 운영체제로 트랩 전달

결과

- 접근 차단
- 프로세스 종료

의미

- 같은 주소공간에 존재하지만, 접근은 완전히 차단
즉, 구조적으로는 공유, 논리적으로는 분리

[4] 페이지 교체: 하드웨어 한계 극복(VMS)

[4-1] 문제 상황: Reference Bit 의 부재

일반적인 페이지 교체 알고리즘(LRU 근사 등)은 다음 정보를 필요로 한다.

- 어떤 페이지가 최근에 사용되었는지(reference bit)
- 어떤 페이지가 오랫동안 사용되지 않았는지

하지만 VAX/VMS 환경에서는 중요한 제약이 있었다.

- Reference Bit(참조 비트)가 없음
- 하드웨어 수준에서 “최근 사용 여부”추적 불가능

결과

- LRU(Least Recently Used) 직접 구현 불가
- 단순 FIFO 는 성능이 매우 나쁨

[4-2] 해결 전략: Segmented FIFO(세그먼트 FIFO)

VMS 는 “단순 FIFO + 완충 구조”로 문제를 해결한다.

1. 기본 구조: RSS 기반 제한

각 프로세스는 다음을 할당받는다.

- RSS(Resident Set Size) -> 프로세스가 메모리에 유지할 수 있는 최대 페이지 수

동장 방식:

- RSS 를 초과하면 가장 먼저 들어온 페이지부터 제거(FIFO)

장점:

- 구현이 매우 단순
- 프로세스별 메모리 사용량 제한 가능

단점:

- 단순 FIFO 는 “자주 쓰는 페이지”도 제거할 수 있음

[4-3] Second-Chance 메커니즘

FIFO의 단점을 해결하기 위해 2차 완충 구조(Second-Chance List)를 도입한다.

두 개의 전역 리스트

- Clean List: 수정되지 않은 페이지 저장
- Dirty List: 수정된 페이지 저장

동작 흐름

1. 어떤 프로세스 P가 RSS 초과
2. 가장 오래된 페이지 제거(FIFO)
3. 제거된 페이지는 바로 삭제되지 않음
 - 수정 X: Clean List
 - 수정 O: Dirty List

페이지 재사용

다른 프로세스가 해당 페이지를 다시 필요로 할 경우

- 디스크 접근 x
- Clean/Dirty List에서 즉시 복구 가능

효과

- 디스크 I/O 감소
- 최근 사용 페이지가 자연스럽게 유지됨
- 결과적으로 LRU와 유사한 동작 효과

완벽한 LRU를 흉내 내는 것이 아니라, 디스크 접근을 줄이는 방향으로 근사

[4-4] 전역 Second-Chance 구조의 의미

이 구조는 단순한 FIFO 를 넘어선 중요한 특징을 가진다.

전역 공유 구조

- Clean/Dirty List 는 모든 프로세스가 공유
- 특정 프로세스만 유리한 구조가 아님

메모리 호그(memory hog) 대응

- 문제: 어떤 프로세스가 메모리를 과도하게 사용하면 전체 성능 저하
- 해결: RSS 제한 + 전역 리스트 균형 유지

결과

- 공정성 + 성능 균형 확보
- 단일 프로세스가 시스템 전체를 망치지 못함

[4-5] 페이지 클러스터링(I/O 최적화)

문제: 작은 페이지의 비효율

- VAX 페이지 크기: 512B(매우 작음)

문제:

- 페이지 하나씩 디스크에 쓰면
 - I/O 오버헤드 과다
 - 디스크 효율 최악

해결: Clustering

VMS 는 페이지를 개별 처리하지 않고 묶어서 처리한다.

- Dirty List 에서 여러 페이지 선택
- 클러스터(cluster) 단위로 디스크 기록

효과

- 디스크 seek 감소
- 한 번에 큰 데이터 전송
- I/O throughput 증가

작은 페이지의 단점을 I/O 배치 최적화로 해결

[5] 성능 최적화: Lazy 전략

VMS 의 가장 중요한 설계 철학 중 하나는 필요할 때까지 아무것도 하지 않는 것이다.

이 전략은 CPU, 메모리, 디스크 I/O 모두에서 성능을 크게 개선한다.

[5-1] Demand Zeroing(요청 시 0 초기화)

1. 기존 방식(비효율적)

- 새로운 메모리 페이지가 필요하면

1. 물리 메모리 할당
2. 즉시 0 으로 초기화
3. 프로세스에 매핑

문제

- 실제로 사용하지 않을 수도 있는데 미리 비용 발생
- 메모리 + CPU 낭비

2. VMS 방식: Demand Zeroing

페이지를 미리 만들지 말고, 접근할 때까지 기다린다.

동작 방식

1. 힙/스택 확장 요청 발생
2. 실제 물리 페이지 할당 x
3. 대신 페이지 테이블에
 - 접근 불가 페이지로 표시
 - 이 페이지는 zero-page 필요 라는 정보 저장
4. 프로세스가 해당 페이지 접근
5. page fault 발생
6. OS 가 처리
 - 물리 페이지 할당
 - 0 으로 초기화
 - 매핑 완료

효과

- 사용하지 않는 페이지는 아예 생성 안함
- 초기화 비용 제거
- 메모리/CPU 절약

0 으로 초기화조차 필요할 때까지 미룬다.

[5-2] Copy-on-Write(COW)

1. 기존 방식(비효율)

- 예 fork()
 - 부모 프로세스 주소 공간 전체 복사
 - 자식 프로세스에 그대로 복제
- 문제:
 - 메모리 크기만큼 복사 비용 발생
 - 대부분은 exec()로 바로 덮어쓰기 -> 낭비

2. VMS 방식: Copy-on-Write

복사하지 말고 공유

동작 방식

- 1 단계: 공유 상태
 - 부모와 자식이 같은 물리 페이지 공유
 - 페이지는 읽기 전용
- 2 단계: 쓰기 발생 시
 - 한쪽 프로세스가 write 시도
 - page fault 발생
- 3 단계: 실제 복사
 - OS가 새로운 물리 페이지 생성
 - 해당 프로세스에만 복사본 연결
 - 다른 프로세스는 기존 페이지 유지

효과

- fork() 거의 즉시 수행 가능
- 실제 복사는 “필요한 경우에만”
- 메모리 사용량 크게 감소

3. COW 의 중요성(Unix 관점)

특히 fork + exec 구조에서 매우 중요하다.

- fork -> 거의 즉시 실행
- exec -> 주소 공간 대부분 덮어씀

따라서 대부분의 복사는 “쓸모 없음”

COW 없으면 완전 비효율, COW 있으면 거의 zero-cost fork

[5-3] Reference Bit 에뮬레이션(하드웨어 없이 LRU 흉내)

문제

- VAX 에는 referece bit 없음
- 최근 사용 여부 알 수 없음
- LRU 구현 불가능

해결: Protection Bit 활용 트릭

- 일부러 접근을 막고, 접근이 발생하면 기록한다.

동작 방식

1. 모든 페이지를 접근 불가(protection bit = no access)로 설정
2. 프로세스가 페이지 접근 시:
 - page fault 발생
3. OS 가 trap 처리
 - 이 페이지 실제로 접근 가능한 페이지인가? 확인
 - 맞으면 다시 정상 권한 부여

결과

- 접근된 페이지만 fault 발생
- fault 기록 = 최근 사용됨

효과

- reference bit 없이 LRU 근사 가능
- 사용되지 않은 페이지 구분 가능

단점

- page fault 발생 -> 오버헤드 큼
- 너무 자주 하면 성능 저하
- 너무 안하면 정보 부족

하드웨어 기능이 없으면 OS 가 fault 로 만들어낸다.